CS54 · DATA COMMUNICATION & NETWORKING · MSRIT · 5TH SEM

CIE-1 STRIKE SHEET

27 questions that have actually been asked — with full 5-mark answers, diagrams and memory hooks

8	76	27	30
PAST PAPERS MINED	QUESTIONS ANALYSED	HIGH-PROBABILITY QS	MARKS TO BEAT

Your CIE-1 portion — read this twice

IN ✓ Unit 1 complete — Section A (data communication, networks, network types, protocol layering, OSI & TCP/IP models) + Section B (HTTP, FTP, SMTP/email, DNS, P2P).

IN ✓ Unit 2 complete — multiplexing/demultiplexing, UDP, Go-Back-N, Selective Repeat, TCP (segment, RTT, RDT, flow control, connection management, congestion control).

IN ✓ Unit 3 only up to IPv4 Addressing — network layer role, IPv4 address & notations, prefix/suffix, classful, classless (CIDR), mask/first/last, block allocation, subnetting.

OUT × Routing algorithms (DV, LS, RIP, OSPF), Unit 4 (data-link control, error control, CRC, CSMA), Unit 5 (LAN, physical layer, multiplexing).

HOW THIS SHEET IS BUILT

Every question here was asked in a real MSRIT paper for this course. The **orange badge** tells you how many times it has appeared and in which CIE-1s. Nothing speculative is included.

Colour code: **blue = Unit 1**, **violet = Unit 2**, **teal = Unit 3**, **amber = memory hook**, **pink = examiner trap**.

HOW TO USE IT IN THE LAST HOURS

1. **Pass 1 (40 min):** read only the question titles + the dashed *Scoring key* box of each card.
2. **Pass 2 (60 min):** work every numerical on paper — Unit 2 RTT and Unit 3 subnetting are guaranteed marks.
3. **Pass 3 (30 min):** hand-draw the 8 starred diagrams once each.
4. **Last 10 min:** page 2 (pattern) + the final two pages (mnemonic vault + formula card).

UNIT 1 · 10 questions

1. Recursive & iterative DNS resolution
2. DNS hierarchy: root, TLD, authoritative
3. DNS resource records & services
4. Cookies and user state
5. Email: SMTP sequence, POP3 vs IMAP
6. Persistent vs non-persistent HTTP
7. HTTP message format + conditional GET
8. TCP/IP suite: 5 layers, logical connections
9. OSI model + address per layer
10. Mesh vs star vs bus topology

+ insurance: web caching, P2P formulas, FTP

UNIT 2 · 9 questions

11. TCP congestion control + AIMD + Reno graph
12. FSM of TCP congestion control
13. Go-Back-N (lost frame) & Selective Repeat
14. RTT, DevRTT, timeout — numerical
15. 3-way handshake, close, client states
16. TCP segment structure & the 8 flags
17. Flow control and rwnd
18. Multiplexing and demultiplexing
19. UDP: why, structure, checksum

+ insurance: TCP sender's 3 events, cumulative ACK sum

UNIT 3 · 8 questions

20. CIDR: mask, N, first & last address
21. Subnetting design, unequal sub-blocks
22. ISP block across many organisations
23. Splitting a block into equal subnets
24. Classful addressing + need for CIDR
25. Classful vs classless addressing
26. Network layer: role, 4 functions
27. IPv4 basics: notations, prefix, class

+ insurance: IPv4 datagram format, fragmentation

Sources: CIE-1 Apr 2025 (CS44) · CIE-1 May 2024 · CIE-1 Jun 2023 (AI42/AD42) · CIE-1 Jun 2021 (CS44) · CIE-2 Jul 2024 · SEE Jan 2020 · Makeup Feb 2020 · SEE Feb 2024, plus Dr. Sangeetha V's CS54 slide decks for Units 1–3.

BRIEFING Decode the paper before you write

30 marks · 1 hour

The exact CIE-1 format (unchanged since 2021)

Element	What it means for you
Max marks 30, duration 1 hr	≈ 2 minutes per mark. A 5-mark answer = 10 min max.
Q1 is compulsory	You cannot dodge it. It always mixes CO1 + CO2 + CO3, i.e. one part from each unit.
Answer Q2 *or* Q3	Read both fully before choosing. Pick the one whose numerical you can finish.
Each question = 3 parts (a, b, c)	Parts carry 2–5 marks. Treat every part as a 5-marker and trim if short.
Bloom levels L2–L4	Verbs used: *Illustrate, Apply, Compare, Analyze, Identify, Justify*. All of them want a diagram or a table, not a paragraph.

TRAP Blooms L2+ means a plain definition scores ~40%. Every single answer in this sheet is built as **definition → structure/steps → diagram or table → one example**. Write all four parts.

What the last two CIE-1 papers actually contained

CIE-1 Apr 2025 (CS44)	M	CO
Q1a Recursive & iterative DNS resolution	2	1
Q1b Go-Back-N with a lost frame + sequence diagram	3	2
Q1c Classful vs classless + **192.168.10.55/26**	3	3
Q2a Cookies for a login session	3	1
Q2b RTT = 170 ms → EstimatedRTT, DevRTT, Timeout	3	2
Q2c Layers of the TCP/IP suite, contribution of each	4	1
Q3a Fields of the TCP segment structure	3	2
Q3b SMTP: A → B when servers differ	3	1
Q3c Significance of each step of the 3-way handshake + close	4	2

READ IT Every single one of those nine is a full card in this sheet. The question bank for CIE-1 is small and it recycles.

Frequency table — the real reason these 27 questions were chosen

Unit 1 topic	×
DNS (hierarchy, recursive/iterative, RRs)	**7**
Electronic mail (SMTP, POP3/IMAP)	**5**
Networks & topologies	3
P2P file distribution	3
TCP/IP suite & layering	3
Persistent / non-persistent HTTP	2
HTTP message format	2
Cookies	2
Web caching	2
OSI model	2

Unit 2 topic	×
TCP congestion control	**7**
Multiplexing / demultiplexing	4
Go-Back-N & Selective Repeat	4
Flow control (rwnd)	4
UDP (choice, structure, checksum)	3
TCP segment structure	3
TCP connection management	3
RTT estimation & timeout	2
Reliable data transfer in TCP	2
(32 questions in 7 papers)	

Unit 3 topic	×
Subnetting / block design	**5**
Classful addressing	2
Classless addressing (CIDR)	1
Mask, first & last address	1
Network layer role & services	1
Beyond your portion, kept as insurance:	
IPv4 datagram format	2
Fragmentation	2

NOTE Unit 3 is **80% numerical**. Addressing sums are the single most reliable marks in the whole paper — they cannot be argued with.

Exam-hall operating procedure

1. **Minute 0–3:** read Q1, Q2, Q3 fully. Circle every numerical. Decide Q2 or Q3 on the strength of its numerical, not its theory.
2. **Do numericals first**, inside the chosen question. They are the fastest full marks and they do not depend on wording.
3. **Every answer starts with a one-line definition**, underlined. Examiners scan for it.
4. **Draw the diagram before writing the prose** — if you run out of time, a labelled diagram still earns 2–3 marks; prose without a diagram does not.
5. **Number your points.** "4 reasons", "3 phases", "5 steps" — lists are what the key is written against.

Marks-per-element budget for a 5-mark answer

Element	Marks
Definition / "what it is" line	1
Labelled diagram or table	2
Numbered steps / fields / reasons	1.5
Worked example or one real protocol name	0.5

NEVER Never leave a part blank. A half-remembered labelled diagram beats an empty space every time.

UNIT 1 Fundamentals + Application Layer

10 questions · CO1 · Forouzan ch.1 + Kurose ch.2

1 Recursive and iterative DNS resolution

ASKED IN BOTH LAST TWO CIE-1S | 5 MARKS | CO1 | DRAW IT

> *"Illustrate how recursive and iterative DNS resolution processes work."*
> CIE-1 Apr 2025 Q1a · CIE-1 May 2024 Q1b

Setup for both: host `cse.nyu.edu` wants the IP of `gaia.cs.umass.edu`. Its local DNS server is `dns.nyu.edu`.

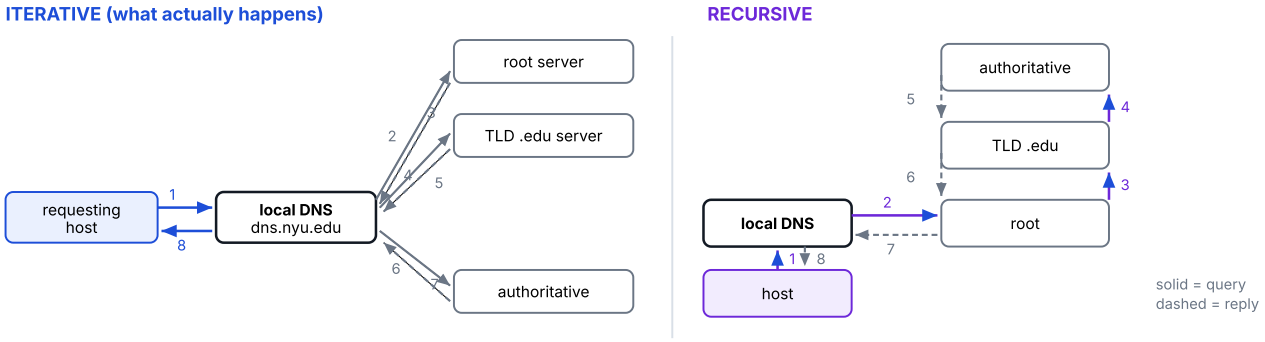
A. Iterative resolution — the usual case (Fig. 2.19)

1. Host → local DNS server: query. *(This first hop is always **recursive**.)*
2. Local → **root** server; root replies with the IP of the **.edu TLD** server.
3. Local → **TLD** server; it replies with the IP of the **authoritative** server `dns.umass.edu`.
4. Local → **authoritative** server; it replies with the IP of `gaia.cs.umass.edu`.
5. Local DNS server → host: the final answer.

Each contacted server says *"I don't know, but ask this server"*, so the **local server does all the work**. Total = **8 messages** (4 queries + 4 replies).

B. Recursive resolution (Fig. 2.20)

host → local → root → TLD → authoritative, **each server querying the next on behalf of the previous one**. The answer travels back down the same chain: authoritative → TLD → root → local → host. The burden shifts **upward** onto the higher-level servers.



Recursive	Iterative
"Find me the **complete answer**." The server that receives the query is responsible for getting the final answer.	"If you don't know, **tell me whom to ask next**." The server returns the best info it has — usually another server's address.
Puts **more load on higher-level** (root/TLD) servers.	Work is done by the **local DNS server**; this is the usual mode for queries leaving the local server.

HOOK **R**ecursive = **R**esponsibility passed upward (one chain up, one chain down). **I**terative = **I**'ll do it myself (local server bounces off each server in turn). In the real Internet it is **hybrid**: host→local is recursive, everything after that is iterative.

SCORING KEY Both diagrams with **numbered arrows** (2) · the 5 iterative steps (1.5) · the recursive chain described (1) · the 8-messages figure and the caching line (0.5). Add: *"DNS caching at the local server lets steps 2–4 be skipped next time, until the TTL expires."*

2 DNS hierarchy: root, TLD and authoritative servers

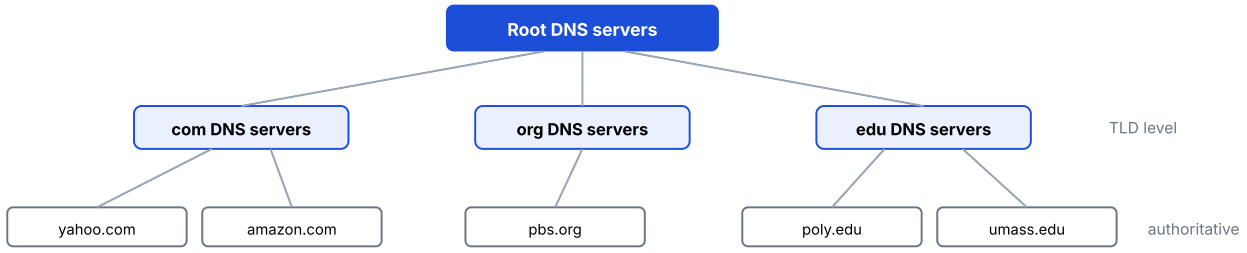
DNS = 7 QUESTIONS, MOST-ASKED TOPIC OF UNIT 1 | 5–6 MARKS | CO1 | DRAW IT

> *"Explain the different hierarchy levels of DNS servers with a neat figure." / "Explain the three classes of DNS servers organised in a hierarchy."*
>
> **Makeup Feb 2020 Q2b · SEE Feb 2024 Q1b**

Definition: DNS is a **distributed, hierarchical database** — no single server holds all the name→IP mappings.

Why not one central server? — 5 problems

1 Single point of failure | **2** Enormous traffic volume | **3** Distant centralised database → delay | **4** Impossible maintenance | **5** Does not scale



Portion of the DNS server hierarchy (slide p. 227) — draw exactly this

Class	Facts to quote	What it gives you
1. Root	1000+ instances worldwide, copies of **13** root servers, run by **12** organisations, coordinated by **IANA**	IP addresses of the **TLD** servers
2. TLD	One server/cluster per top-level domain: com, org, net, edu, gov + country codes uk, fr, ca, jp. **Verisign** runs .com; **Educause** runs .edu	IP addresses of the **authoritative** servers
3. Authoritative	Every organisation with public hosts keeps its own records here (own server or a provider's). Usually a **primary + secondary** backup	The **actual IP address** of the host
+ Local DNS	**Not strictly part of the hierarchy.** Every ISP/campus has one. It is the **first server contacted** and acts as a **proxy** into the hierarchy.	

HOOK "**R**oot **T**ells, **T**LD **T**ells, **A**uthoritative **A**nswers." Only the authoritative server knows the real IP — the other two only hand you the next address. **Numbers to quote: 13 root servers · 12 organisations · IANA.**

SCORING KEY The tree diagram (2) · the 3 classes with what each returns (2) · local DNS server as proxy (0.5) · DNS caching + TTL (0.5). Close with the example: *"to resolve www.amazon.com → ask root (gets .com TLD) → .com TLD (gets amazon.com authoritative) → authoritative (gets the IP)."*

3 DNS resource records and the services DNS provides

ASKED 3× | 4–8 MARKS | CO1

> *"Discuss the various DNS resource records with an example." / "Explain the services provided by DNS and identify the resource records used to provide these services."*
>
> **CIE-1 Jun 2023 Q1c · Makeup Feb 2020 Q1c · SEE Jan 2020 Q2c**

Definition: DNS servers store **resource records (RRs)**. Each RR is a **4-tuple: (Name, Value, Type, TTL)**. **TTL** says when the record must be removed from a cache; the meaning of *Name* and *Value* depends on *Type*.

Type	Name → Value	Example
A	hostname → its IPv4 address	`(relay1.bar.foo.com, 145.37.93.126, A)`
NS	domain → hostname of its authoritative DNS server	`(foo.com, dns.foo.com, NS)`
CNAME	alias → canonical hostname	`(foo.com, relay1.bar.foo.com, CNAME)`
MX	alias → canonical name of the **mail** server	`(foo.com, mail.bar.foo.com, MX)`

Service	RR used
Hostname → IP translation — turns www.someschool.edu into the IP the browser connects to	**A** (AAAA for IPv6)
Host aliasing — an easy alias for an ugly canonical name (www.company.com for server1.company.com)	**CNAME**, then **A**
Mail-server aliasing — a simple mail domain (bob@company.com) for a complicated server name	**MX**, then **A**
Load distribution — a busy site has many servers; DNS returns the whole set of IPs and **rotates their order**	several **A** records
Finding the right server — delegation from TLD to the organisation's own server	**NS**

HOOK **"A NaMe Can Mail"** → **A** (address), **NS** (name server), **CNAME** (canonical name), **MX** (mail exchange). And the tuple order is **N-V-T-T**: Name, Value, Type, TTL.

TRAP If the question adds *"discuss inserting records into the DNS database"*: a startup registers **networkutopia.com** with an **ICANN-accredited registrar**, gives it the names + IPs of its primary & secondary authoritative servers, and the registrar inserts **two** records into the **.com TLD servers** — `(networkutopia.com, dns1.networkutopia.com, NS)` and `(dns1.networkutopia.com, 212.212.212.1, A)`. The company then adds its own A record (web server) and MX record (mail server).

SCORING KEY 4-tuple definition (1) · the 4 RR types with one example each (2) · the services table (1.5) · TTL/caching line (0.5).

UNIT 1 Fundamentals + Application Layer

continued · Q4 – Q7

4 Cookies — keeping user state over stateless HTTP

CAME IN CIE-1 APR 2025 | 5 MARKS | CO1 | DRAW IT

> *"How would a web server use cookies to remember a user's login session across multiple visits? Illustrate with an example." / "Describe user-server interaction using cookie technology."*
>
> CIE-1 Apr 2025 Q2a · SEE Feb 2024 Q2b

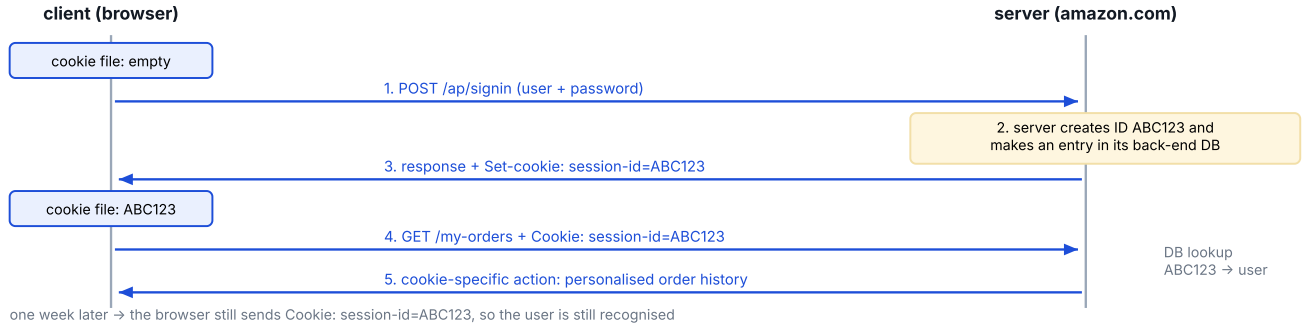
Why cookies exist: HTTP is a **stateless** protocol — the server forgets a client the moment it replies. Cookies let a site **identify a user across requests and across visits** (login sessions, shopping carts, recommendations).

The 4 components (always write all four)

1. A `Set-cookie:` header line in the HTTP **response**.
2. A `Cookie:` header line in subsequent HTTP **requests**.
3. A **cookie file** on the user's machine, managed by the browser.
4. A **back-end database** at the website.

Example — Amazon login session

1. User logs in: browser sends `POST /ap/signin` with username and password.
2. Server creates a **unique ID** (e.g. `session-id = ABC123`), stores it in its **database** against this user, and replies with `Set-cookie: session-id=ABC123`.
3. The browser saves it in its **cookie file** (`amazon → ABC123`).
4. On every later request ("My Orders") the browser **automatically** adds `Cookie: session-id=ABC123`.
5. The server looks the ID up in its database, recognises the logged-in user, and returns **personalised content** — no password asked again.
6. A week later the browser **still sends the cookie** (until it expires), so the user is still recognised.



Kurose-style cookie timeline — client, server, cookie file and back-end database

HOOK Four components = **"Set it, Send it, Store it, Search it"** → **Set-cookie** header · **Cookie** header · cookie **file** in the browser · back-end **database**. Cookies make a *stateless* protocol behave *statefully*.

SCORING KEY "HTTP is stateless" opening line (1) · the 4 components (1.5) · the timeline diagram (1.5) · the numbered Amazon example (1).

5 Electronic mail — SMTP sequence, SMTP vs HTTP, POP3 vs IMAP

EMAIL = 5 QUESTIONS · CAME IN CIE-1 APR 2025 | 5–8 MARKS | CO1 | DRAW IT

> *"Illustrate the sequence of events when SMTP is used to send an email from User A to User B, where the servers are different."*
>
> CIE-1 Apr 2025 Q3b · SEE Feb 2024 Q1a (compare SMTP/FTP/HTTP) · SEE Jan 2020 Q1c (POP3 vs IMAP)

3 major components of email: user agents (Outlook, Gmail app — compose/read/reply/forward), **mail servers** (hold each user's *mailbox* + an *outgoing message queue*), and **SMTP** (the protocol that pushes the mail).



Fig. 2.16 — Alice's agent → Alice's mail server → SMTP → Bob's mail server → Bob's agent

The 6 steps to write

1. User A writes a message to B's address in A's **user agent**.
2. The user agent sends it to **A's mail server**, where it is placed in the **outgoing message queue**.
3. The **SMTP client side** on A's server opens a **TCP connection to port 25** on B's mail server.
4. After **SMTP handshaking** (sender and recipient addresses are agreed), the client sends A's message over that TCP connection.
5. B's mail server (**SMTP server side**) places the message in **B's mailbox**.
6. User B runs their user agent and reads it — fetched with **POP3, IMAP or HTTP**.

SMTP transcript (write it if asked)

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 closing connection
```

POP3 vs IMAP

	POP3	IMAP
Port	**110**	**143**
Job	Authorisation + **download**, usually **deleting** from the server	**Keeps** mail on the server and lets the user manage it there
Mode	Offline	Online / synced on every device
Folders	Local machine only	Folders **on the server**
State	No state across sessions	Keeps state (folders, read/unread)

POP3's 3 phases: Authorization (`user`, `pass`) → **Transaction** (`list`, `retr`, `dele`) → **Update** (on `quit` the marked mails are deleted). Server replies +OK / -ERR.

SMTP vs HTTP

	HTTP	SMTP
Direction	**Pull** — client pulls objects	**Push** — sender pushes the file
Encoding	No restriction	**7-bit ASCII only**
Objects	Each object in its own response	**All objects in one message**
Port	80 / 443	25

HOOK SMTP's five client commands: **H-M-R-D-Q** → "**H**ello **M**ail **R**eceiver, **D**ata **Q**uit" = HELO, MAIL FROM, RCPT TO, DATA, QUIT. Ports: **SMTP 25 · POP3 110 · IMAP 143** ("25 sends, 110 takes away, 143 keeps").

SCORING KEY The 5-box diagram (1.5) · the 6 numbered steps (2) · ports 25/110/143 named (0.5) · "SMTP is direct — no intermediate mail servers" + the retry line (1).

UNIT 1 Fundamentals + Application Layer

continued · Q6 – Q8

6 Persistent vs non-persistent HTTP (with the RTT count)

ASKED 2× | 5–6 MARKS | CO1 | NUMERICAL INSIDE

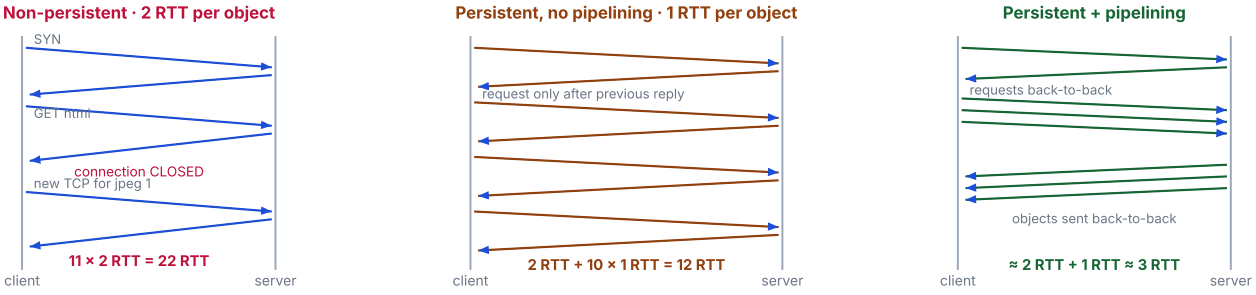
> *"Explain HTTP with non-persistent connections with an example." / "Differentiate between persistent and non-persistent TCP connections used in HTTP."*
>
> SEE Jan 2020 Q1a · Makeup Feb 2020 Q1a

Non-persistent: each request/response pair uses a **separate TCP connection**, which the server **closes** after sending the object. Default in **HTTP/1.0**.
Persistent: one TCP connection is reused for many requests; the server keeps it open for a while. Default in **HTTP/1.1**.

Standard example — 1 HTML page + 10 JPEGs = 11 objects

1. **1a.** Client opens a TCP connection to `www.someSchool.edu` on **port 80**. **1b.** The server, waiting on port 80, accepts it.
2. Client sends an HTTP request with the path `/someDepartment/home.index` into its socket.
3. Server reads it, puts the object in an HTTP response and sends it.
4. Server **closes** the TCP connection.
5. Client receives the HTML, displays it, parses it and finds the **10 JPEG references**.
6. Steps 1–5 **repeat for each JPEG** → **11 TCP connections in total**.

```
Response time per object = 2 RTT + transmit time   (1 RTT to set up TCP, 1 RTT to request & start receiving)
```



The same 11-object page under the three modes — this comparison *is* the answer

	Non-persistent	Persistent
Connections	New TCP connection per object, closed after each response	One TCP connection reused for many requests
Default in	HTTP/1.0	HTTP/1.1
Delay/object	2 RTT + transmit	≈1 RTT without pipelining; back-to-back with pipelining
Server load	TCP buffers & variables allocated **for every object** — heavy for busy servers	Allocated **once per client**
Types	—	Without pipelining (connection sits idle) · with pipelining (requests sent as soon as references are found)

HOOK The **2 shortcomings of non-persistent** = **"Buffers and Delay"**: (1) new TCP buffers/variables per object, (2) 2 RTT per object. Both are fixed by persistence.

SCORING KEY Both definitions with the HTTP version (1) · the 6-step 11-object walk-through (1.5) · the RTT timing diagram (1.5) · the 22 / 12 / 3 RTT arithmetic (1).

7 HTTP message format + conditional GET

ASKED 2×, BOTH IN CIE-1S | 5–6 MARKS | CO1

> *"Draw the general format of an HTTP request message and give an example of request and response messages using conditional GET." / "Interpret the meaning of each line of the HTTP message below."*
>
> CIE-1 Jun 2023 Q1a · CIE-1 May 2024 Q3a

1. General format of a request message

```
method sp URL sp version cr lf      ← request line
header field name: sp value cr lf     ← header lines
        ...                          (one or more)
header field name: sp value cr lf
cr lf                                ← blank line
entity body                          ← empty for GET
```

Reading a real request, line by line

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

- **Request line** — 3 fields: **method** GET (browser is asking for an object), **URL** (path of the object), **version**.
- **Host:** names the host the object resides on. Needed because one server can host many sites; used by web proxy caches.
- **Connection: close** — the browser asks the server to close the TCP connection after this object (it does **not** want persistence).
- **User-agent:** the browser type. The server can send different versions of an object to different browsers.
- **Accept-language: fr** — the user prefers the French version; if none exists the server sends its default.

Always add: a blank line (extra CRLF) ends the headers, and the entity body is empty for GET (used for POST).

2. Conditional GET

A **conditional GET** = a GET carrying an `If-Modified-Since:` header. It asks the server to send the object **only if it has changed** since that date. Used by **web caches** to make sure a stored copy is not stale.

First request & response — the cache stores the object *and* its Last-Modified date:

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com

HTTP/1.1 200 OK
Date: Sat, 3 Oct 2015 15:39:29
Server: Apache/1.3.0 (Unix)
Last-Modified: Wed, 9 Sep 2015 09:23:24
Content-Type: image/gif
(data data data ...)
```

A week later the cache re-checks:

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-modified-since: Wed, 9 Sep 2015 09:23:24

HTTP/1.1 304 Not Modified
Date: Sat, 10 Oct 2015 15:39:29
Server: Apache/1.3.0 (Unix)
(empty entity body)
```

304 Not Modified tells the cache its copy is still valid — the object is **not re-sent**, saving bandwidth.

Status codes to memorise

Code	Phrase	Meaning
200	OK	Request succeeded, object follows
301	Moved Permanently	New URL in the `Location:` header
400	Bad Request	Server couldn't understand it
404	Not Found	Document doesn't exist
505	HTTP Version Not Supported	Version not supported

HOOK A response message has **3 sections: status line → header lines → entity body**, and the status line has **3 fields: version, status code, status phrase**. Methods: **GET, POST, HEAD, PUT, DELETE** (+ TRACE, CONNECT, OPTIONS).

SCORING KEY The labelled general format box (1.5) · line-by-line meaning of 5 headers (1.5) · the conditional GET pair with **304 Not Modified** (2).

UNIT 1 Fundamentals + Application Layer

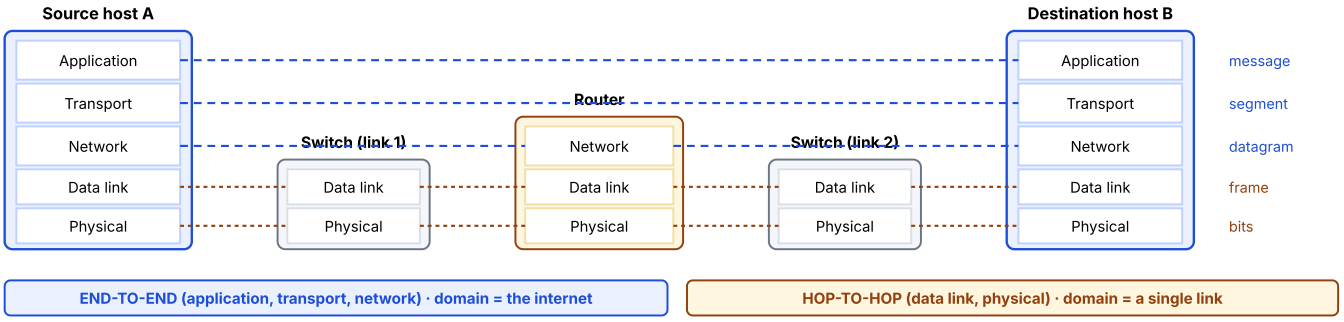
continued · Q8 – Q10 + insurance

8 TCP/IP protocol suite — the 5 layers and their logical connections

ASKED 3× INCL. CIE-1 APR 2025 | 5 MARKS | CO1 | DRAW IT

> *"Analyze the different layers in the TCP/IP protocol suite. How does each layer contribute to the overall communication process?" / "Explain with a neat diagram the logical connections between the layers of the TCP/IP protocol suite."*
>
> CIE-1 Apr 2025 Q2c · CIE-1 Jun 2023 Q2a · CIE-1 Jun 2021 Q1a

Definition: TCP/IP (built by **DARPA in the 1970s for ARPANET**) is a **hierarchical protocol suite** — each upper-level protocol is supported by one or more lower-level ones. Originally **4 software layers on hardware**; today modelled as **5 layers**.



Layers involved: HOST = all 5 · ROUTER = 3 (network, data link, physical) · LINK-LAYER SWITCH = 2 (data link, physical)
Below each layer the two objects are identical. Switches are not drawn in Fig. 7.x because they do not change the object. A "hop" is a host or a router.

Fig. 3.x — logical (dashed) connections between peer layers of the TCP/IP suite

Layer	Contribution to the communication	Object	Address
5. Application	Process-to-process communication (request/response). The user's message is created here. HTTP, SMTP, DNS, FTP.	Message	hostname / URL
4. Transport	**End-to-end** logical connection; encapsulates the message into a **segment (TCP)** or **user datagram (UDP)**; delivers to the right process; TCP adds reliability.	Segment	Port (16 bit)
3. Network	**Host-to-host** delivery of datagrams; adds source and destination IP; routers choose the best route. IP is connectionless, best-effort.	Datagram	IP (32 bit)
2. Data link	Moves frames **hop to hop** across each link; adds the MAC of the next device; detects errors on the link.	Frame	MAC (48 bit)
1. Physical	Carries the bits as electrical / optical / wireless signals over the medium.	Bits	—

Close with the slide example: a WhatsApp "Hello Friend" goes **down** the 5 layers on the sender, through routers (which use layers 1–3 only), and **up** the 5 layers on the receiver. **If the question adds "services provided by the network layer":** logical addressing, routing, packet forwarding, internetworking — and IPv4 is connectionless and best-effort.

HOOK **Host 5, Router 3, Switch 2.** Objects top-down: **M-S-D-F-B** → "**M**y **S**ister **D**raws **F**unny **B**irds" = Message, Segment, Datagram, Frame, Bits.

SCORING KEY The 5-device diagram with dashed peer links (2) · end-to-end vs hop-to-hop with their domains (1) · one line per layer (1.5) · the object names under each layer (0.5).

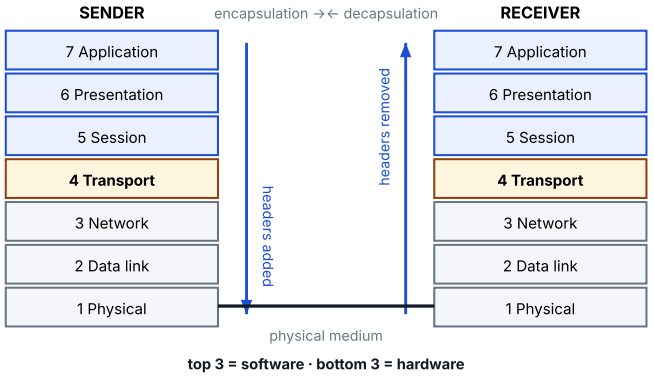
9 OSI model — 7 layers, encapsulation, and the address used at each layer

ASKED 2× | 4–6 MARKS | CO1 | DRAW IT

> *"Explain the layering functionality of the OSI model." / "What types of addresses (identifiers) are used in each of these layers? (i) Application (ii) Network (iii) Transport (iv) Data link."*
>
> SEE Feb 2024 Q2a · CIE-1 May 2024 Q2a

Definition: OSI (**Open Systems Interconnection**) is a **reference model published by ISO** (the organisation, founded 1947) in the **late 1970s**. It is a **model, not a protocol** — a 7-layer framework for designing flexible, robust and interoperable architecture. An **open system** is a set of protocols that lets any two different systems communicate *regardless of their underlying architecture*.

#	Layer	What it does
7	**Application**	Where the user interacts with software; produces the data. File transfer, email, web, virtual terminal.
6	**Presentation**	**Translation** to a uniform format, **encryption/decryption**, **compression**. e.g. SSL.
5	**Session**	Establish/use/end a session; **dialog control** (who sends, when, how long); token management & synchronisation; authentication & authorisation.
4	**Transport**	**Service-to-service** delivery. Splits long messages into packets and reassembles them in order; error recognition and recovery (ACK + resend).
3	**Network**	**End-to-end** delivery. Translates logical names/addresses to physical; determines the route; switching, routing, congestion control.
2	**Data link**	**Hop-to-hop** delivery. Packages raw bits into **frames**; reliable frame transmission (waits for ACK, retransmits).
1	**Physical**	Transmits raw **bits** over the medium; defines cables, connectors and the transmission technique.



Encapsulation down, decapsulation up

HOOK Top→bottom: **"All People Seem To Need Data Processing"**. Bottom→top: **"Please Do Not Throw Sausage Pizza Away"**. Top 3 = **software** layers, bottom 3 = **hardware** layers, **transport = the heart** of OSI.

Addresses used at each layer

Layer	Identifier	Example
Application	Application-specific **name**: hostname/URL or email address	`www.someschool.edu` `alice@crepes.fr`
Transport	**Port number, 16 bits** — identifies the **process**	`80, 443, 53, 25`
Network	**Logical (IP) address, 32 bits** — identifies the host globally, **unchanged** end to end	`192.168.1.10`
Data link	**Physical (MAC) address, 48 bits** on the NIC — **changes at every hop**	`00:1A:2B:3C:4D:5E`

TRAP · ALSO ASKED **OSI vs TCP/IP: session and presentation are missing** from TCP/IP — its application layer = OSI application + presentation + session. **2 reasons:** (1) TCP/IP has more than one transport protocol and some session functions already live in them; (2) the application layer is not one piece of software, so an app that needs session/presentation functions builds them in itself. **Why OSI failed — 3 reasons:** completed **too late** (TCP/IP already deployed), some layers were **never fully defined**, and **poor performance** when implemented.

SCORING KEY "Model, not a protocol" + ISO ≠ OSI (0.5) · one line per layer (2) · the encapsulation/decapsulation figure (1.5) · the address table if asked (1).

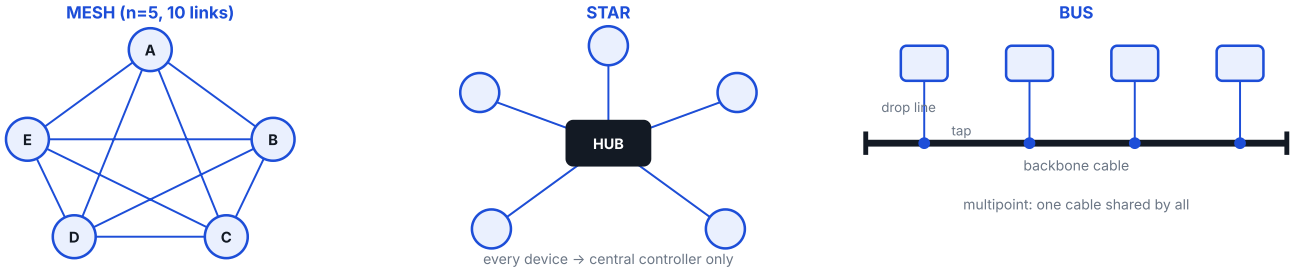
10 Topologies — mesh vs star, and bus (advantages, disadvantages, applications)

3 CIE-1 APPEARANCES: 2024, 2023, 2021 | 5 MARKS | CO1 | DRAW IT

> *"Consider five devices in a mesh topology. (i) How many cables are required? (ii) Draw the topology for 5 nodes. (iii) Compare it with star topology." / "Discuss the advantages, disadvantages and applications of bus topology."*
>
> CIE-1 May 2024 Q1a · CIE-1 Jun 2023 Q3b · CIE-1 Jun 2021 Q3b

Topology = the geometric representation of the relationship of all the links and nodes to one another. **4 basic topologies: mesh, star, bus, ring.**

```
Mesh links = n(n − 1) / 2   →   n = 5 gives 5 × 4 / 2 = 10 cables · ports per device = n − 1 = 4
```



	Mesh	Star
Connection	Dedicated point-to-point link to **every** other device	Point-to-point link only to a **central hub/switch**; devices not directly linked
Links for n=5	**n(n−1)/2 = 10**	**n = 5**
Ports/device	n − 1 = 4	1
Failure	**No single point of failure**; self-healing, traffic re-routes	**Hub fails → whole network down**; one device failing affects only itself
Traffic	Direct between the two devices	All traffic relayed by the hub
Cost/setup	Expensive, heavy cabling, complex	Cheap, easy to set up, expand and manage
Applications	Military communication, city public Wi-Fi	Home/office Wi-Fi, smart-home automation

Bus topology (the 2021 CIE-1 question)

Bus is multipoint: one long cable acts as a **backbone** linking all devices. A **drop line** connects a device to the main cable; a **tap** is the connector that splices into the cable or punctures its sheathing to touch the metal core.

Advantages — easy and cheap to install (less cable than star) · suitable for small networks · good for **broadcasting** (radio, TV, sensors).

Disadvantages — backbone failure brings down the whole network · **collisions** increase as devices are added · limited scalability, each extra device weakens the signal.

Applications — traditional cable TV networks, early Ethernet LANs.

Topology	Connection	Main weakness
Mesh	P2P, everyone to everyone	Expensive, complex
Star	P2P to a hub	Hub failure
Bus	Multipoint backbone	Backbone failure, collisions
Ring	P2P to 2 neighbours	One node breaks the ring

HOOK "**My Star Bus Rings**" = Mesh, Star, Bus, Ring. Mesh & Star & Ring are **point-to-point**; only **Bus is multipoint**. Devices in a ring each carry a **repeater** that regenerates the bits meant for others.

SCORING KEY The formula with the substitution (1) · both diagrams drawn (1.5) · the comparison table with ≥4 rows (2) · applications line (0.5).

+ Unit 1 insurance — three topics that are cheap to carry

WEB CACHING 2× · P2P 3× · FTP **READ ONCE**

Web caching / proxy server

A **web cache (proxy server)** sits between clients and origin servers and keeps copies of recently requested objects. The browser is configured so **all requests go to the cache first**. The cache is **both a server** (to the browser) **and a client** (to the origin server).

4 steps: (1) browser opens a TCP connection to the cache and sends the request; (2) **hit** → cache returns the object; (3) **miss** → cache opens a TCP connection to the origin server and fetches it; (4) cache stores a copy and forwards one to the browser.

3 benefits: lower response time (cache is near the client) · less traffic on the institution's access link (no costly upgrade) · less web traffic on the Internet overall.

The numerical: LAN 100 Mbps, access link 15 Mbps, objects 1 Mbit, 15 req/s, Internet delay 2 s.
Access-link traffic intensity = (15 × 1)/15 = **1** → delay grows without bound. LAN intensity = 15/100 = 0.15, negligible.
Option 1 upgrade link to 100 Mbps → intensity 0.15, response ≈ 2 s, but **costly**.
Option 2 install a cache, hit rate 0.4 → 40% served from the LAN in ~10 ms, only 60% use the access link (intensity 0.6).
Average delay = 0.4 × 0.01 + 0.6 × 2.01 ≈ 1.2 s — faster than the upgrade *and* much cheaper.

P2P vs client-server file distribution

Notation: file size **F**, **N** peers, server upload $\mathbf{u_s}$, peer *i* upload $\mathbf{u_i}$, slowest download $\mathbf{d_{min}}$.

$$D_{cs} = \max\{ NF/u_s , F/d_{min} \}$$

NF/u_s: the server alone must upload **N copies**. F/d_{min}: the slowest client must still download F.

$$D_{P2P} = \max\{ F/u_s , F/d_{min} , NF/(u_s + \Sigma u_i) \}$$

F/u_s: the server must upload at least **one** copy. The third term: all NF bits must be uploaded by **everybody together**.

Conclusion to write: D_{cs} grows **linearly with N** (the server's upload link is the bottleneck — every new client adds load but no capacity). D_{P2P} grows **much more slowly** because P2P is **self-scaling**: more peers = more upload capacity. Sketch distribution time vs N: a rising straight line for client-server, a curve that **flattens** for P2P.

Worked: F = 15 Gbits = 15,000 Mbits, N = 100, u_s = 30 Mbps, d_{min} = 2 Mbps.
D_{cs} = max(1,500,000/30, 15,000/2) = max(50,000 , 7,500) = **50,000 s**.
u = 300 kbps → u_s+Nu = 60 Mbps → 25,000 s → D_{P2P} = **25,000 s**.
u = 700 kbps → 100 Mbps → **15,000 s**. u = 2 Mbps → 230 Mbps → 6,522 s, so D_{P2P} = **7,500 s** (limited by d_{min}).

UNITS 300 Kbps = 0.3 Mbps. Use 1 Gbit = 1000 Mbits.

FTP

FTP uses **two parallel TCP connections**:

	Control	Data
Job	Commands & replies	The actual file
Port	**21**	**20**
Lifetime	Open for the **whole session**	Opened and closed **per file**

Commands (ASCII uppercase): USER, PASS, LIST, RETR (download), STOR (upload), DELE, MKD/RMD, CWD/CDUP, PWD, RNFR/RNTO, PORT/PASV, TYPE (A/E/I), STRU (F/R/P), MODE (S/B/C), ABOR, QUIT.

FTP is the only one of the three that **keeps session state** (logged-in user, current directory). HTTP is stateless; SMTP keeps none.

HOOK "**21 talks, 20 walks**" — port 21 carries the conversation, port 20 carries the file.

SMTP vs FTP vs HTTP in one line each

SMTP push, port 25, 7-bit ASCII, all objects in one message.
FTP both ways, ports 21+20, keeps state, one data connection per file.
HTTP pull, port 80/443, stateless (cookies add state), each object in its own response.

UNIT 2 Transport Layer

9 questions · CO2 · Kurose ch.3 · 32 past questions

11 TCP congestion control — the 4 mechanisms, AIMD and the Reno graph

7 QUESTIONS — THE SINGLE MOST-ASKED TOPIC IN THE SYLLABUS | 5–8 MARKS | CO2 | DRAW IT

> *"Justify: TCP congestion control is referred to as AIMD." / "What is fast recovery? Draw a neat graph showing the three phases of congestion control in TCP Reno." / "Explain (i) congestion avoidance (ii) fast recovery." / "Explain the approaches to congestion control."*
>
> SEE Feb 2024 Q4a, Q3b, Q3c · SEE Jan 2020 Q4a · Makeup Feb 2020 Q3c, Q4a · CIE-1 May 2024 Q3c

Framework — the 3 questions classic TCP answers

1. **How does the sender limit its rate?** With the **congestion window cwnd** — the amount of unacknowledged data allowed in the network at once. Data in flight = **min(cwnd, rwnd)**.
2. **How does it detect congestion?** From **packet loss** — a retransmission **timeout**, or **3 duplicate ACKs** — and from rising delay (longer RTTs).
3. **What algorithm changes the rate? Slow start → congestion avoidance → fast retransmit → fast recovery.**

#	Mechanism	What happens	cwnd behaviour
1	**Slow start**	Runs at connection start or after a timeout. TCP doesn't know the available bandwidth, so it begins at **cwnd = 1 MSS** and increases cwnd by **1 MSS per ACK**. Ends when **cwnd ≥ ssthresh**.	**Exponential**: 1, 2, 4, 8 — doubles every RTT
2	**Congestion avoidance**	Begins once cwnd reaches ssthresh. The rate is raised gently so the network isn't overloaded: **cwnd += 1 MSS per RTT** (per new ACK: cwnd = cwnd + MSS × MSS/cwnd).	**Linear**: 8, 9, 10 ...
3	**Fast retransmit**	If one packet is lost, later packets still arrive, so the receiver keeps re-sending duplicate ACKs for the missing one. After **3 duplicate ACKs** the sender retransmits **immediately**, without waiting for the timeout.	cwnd unchanged by itself
4	**Fast recovery**	3 duplicate ACKs mean **only one** segment is probably missing, so TCP does **not** collapse to 1 MSS. Steps: retransmit the missing segment; **ssthresh = cwnd/2**; **cwnd = ssthresh + 3 MSS**. Each further duplicate ACK: cwnd += 1 MSS. On the new ACK: **cwnd = ssthresh** → congestion avoidance. On a timeout: → slow start, cwnd = 1 MSS.	**Halved**, not reset

Why it is called AIMD (write this if the question says "justify")

1. Additive increase. In congestion avoidance cwnd rises by **1 MSS per RTT** — linear growth (slide: 8 → 9 → 10). TCP keeps probing upward until a loss shows the network is full.

2. Multiplicative decrease. On **3 duplicate ACKs** the window is **halved**: ssthresh = cwnd/2, cwnd = ssthresh + 3 MSS (Reno). On a **timeout**: ssthresh = cwnd/2 and cwnd falls to **1 MSS**. The decrease is a **multiple (×½)** of the current window, not a fixed amount.

3. Result — the sawtooth. cwnd climbs linearly, drops by half at each loss, climbs again.

4. Why it works. The gentle increase avoids *causing* congestion; the sharp decrease *relieves* it quickly; and competing connections using AIMD **converge to a fair share** of the link.

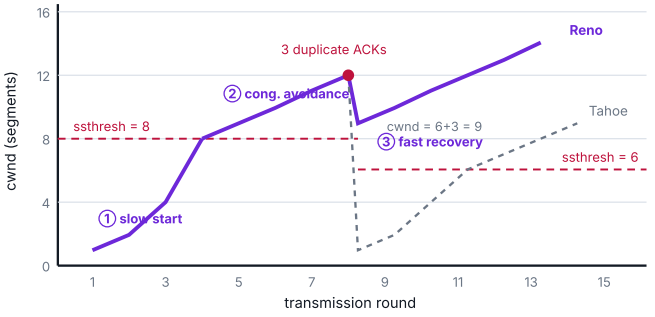


Fig. 3.53 — cwnd per transmission round. Reno (solid) vs Tahoe (dashed)

Reading the graph out loud (examiners love this): initial ssthresh = 8. Slow start 1 → 2 → 4 → 8. At cwnd = 8 = ssthresh, congestion avoidance adds 1 per round up to **12**. At round 8 a triple duplicate ACK signals loss → new **ssthresh = 12/2 = 6**. **Tahoe** drops to cwnd = 1 and slow-starts again; **Reno** goes to ssthresh + 3 = **9** and continues linearly.

HOOK **S-A-R-R** = "**S**tart slow, **A**void trouble, **R**esend fast, **R**ecover." And remember: **slow start is exponential** — it is "slow" only in where it *starts*, not in how it grows.

If asked: "approaches to congestion control"

1. End-to-end. The network layer gives **no explicit support**. End systems **infer** congestion from packet loss (timeout or duplicate ACKs) and rising delay. This is classic TCP — cwnd + the four algorithms.

2. Network-assisted. Routers give **explicit feedback**. Example **ECN**: the congested router sets two ECN bits in the IP Type-of-Service field; the receiver echoes it using the **ECE flag** in the TCP

If asked: "fairness in TCP"

Fairness: if K TCP connections share a bottleneck of rate R, each should get **R/K**. For two connections with throughputs x and y: the **equal-share line is x = y**; **full utilisation is x + y = R**. Both additively increase (moving up at 45°); when x + y exceeds R the router drops packets and both **halve**, which halves the difference between them. Repeating, they **converge to R/2 each** whatever the starting point. **AIMD is what produces fairness.**

ACK; the sender reduces its rate. Congestion is signalled **without dropping packets**.

One-line comparison: end-to-end must wait for a loss or delay to react; network-assisted warns earlier.

Problems in sharing: (1) **UDP is not fair** — multimedia apps don't back off and push TCP out; (2) **parallel connections** — a browser opening 9 connections against 1 gets ~9/10 of R; (3) the model assumes **equal RTTs** — shorter-RTT connections grow faster and take more bandwidth.

SCORING KEY cwnd definition + the two congestion signals (1) · all 4 mechanisms named with their cwnd rule (2) · the Reno/Tahoe graph with both ssthresh dashed lines labelled (2) · the AIMD sawtooth justification (1) · fairness or ECN if asked (2).

12 FSM of TCP congestion control

ASKED 2× (INCL. CIE-1 MAY 2024) · 5–6 MARKS · CO2 · PURE DIAGRAM

> *"Sketch the finite state machine (FSM) for the TCP congestion control mechanism." / "Draw the neat FSM diagram of TCP congestion control."*
>
> **CIE-1 May 2024 Q3c · Makeup Feb 2020 Q4a**

Three states; every arrow is labelled **event / action**. Entry point: `cwnd = 1 MSS, ssthresh = 64 KB, dupACKcount = 0`.

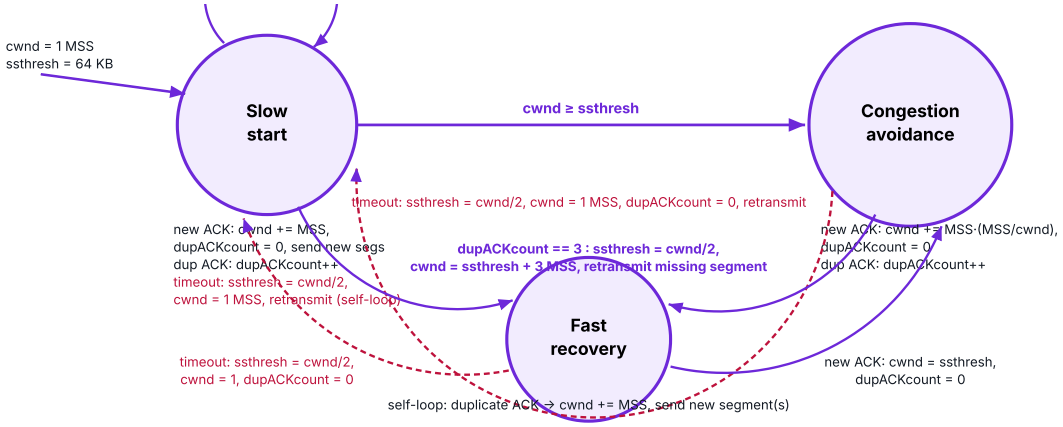


Fig. 3.51 — FSM description of TCP congestion control

HOOK Three states, three exits. **Every timeout arrow goes back to Slow start** (cwnd = 1). **Every triple-dup-ACK arrow goes to Fast recovery** (cwnd = ssthresh + 3). **Only "cwnd ≥ ssthresh" and "new ACK" move you rightwards** into congestion avoidance. Draw the three circles first, then fill arrows in that order and you cannot forget one.

SCORING KEY Three labelled circles (1.5) · entry arrow with cwnd=1, ssthresh=64 KB (0.5) · all six transitions labelled event/action (2.5) · two-line explanation of each state: slow start = exponential, congestion avoidance = linear, fast recovery = keep sending after 3 dup ACKs (1.5).

UNIT 2 Transport Layer

continued · Q13 – Q15

13 Go-Back-N with a lost frame, and the Selective Repeat window proof

ASKED 4× · CAME IN CIE-1 APR 2025 AND MAY 2024 | 5–8 MARKS | CO2 | DRAW IT

> *"Apply the concept of Go-Back-N to a scenario where a data frame is lost during transmission. Use a sequence diagram to show how the protocol handles it."*
>
> CIE-1 Apr 2025 Q1b · CIE-1 May 2024 Q1c · SEE Feb 2024 Q3a · CIE-1 Jun 2021 Q2a (SR)

1. The idea

GBN is a reliable-data-transfer (**ARQ**) protocol. The sender may send up to **N** packets without waiting for ACKs. If a packet is lost, the sender **goes back** and retransmits that packet **and every packet after it**.

2. Sender

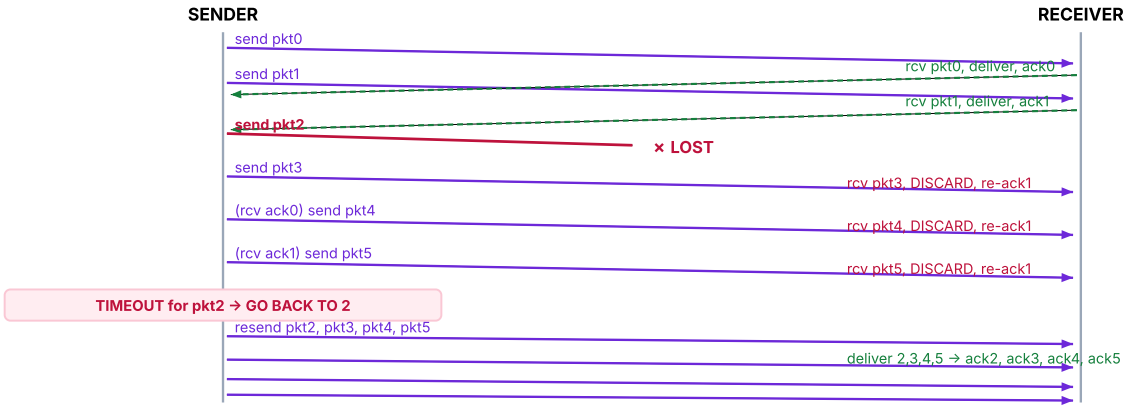
Keeps a window of size N with **base** (oldest unACKed packet) and **nextseqnum** (next to send). **One timer** runs, for the **oldest unACKed** packet. Uses **cumulative ACKs**: "ACK n" means *all* packets up to n arrived. On timeout it resends base … `nextseqnum−1`.

3 events (extended FSM): *rdt_send(data)* → if nextseqnum < base + N, make_pkt & send, start timer if base == nextseqnum, nextseqnum++; else refuse (window full). *ACK received (uncorrupted)* → base = acknum + 1; if base == nextseqnum stop the timer else restart it; a corrupt ACK is ignored. *Timeout* → restart timer and resend all unACKed packets.

3. Receiver

Accepts **only the expected in-order packet** (delivers it and ACKs it, expectedseqnum++). Any **out-of-order or corrupted** packet is **discarded** and the ACK of the **last in-order** packet is re-sent. **The receiver has no buffer** — that is the whole difference from Selective Repeat.

4. Sequence diagram — window N = 4, packet 2 lost



GBN, N = 4, packet 2 lost — the exam wants exactly this picture

5. Drawbacks (needed for 8 marks)

- Correctly received packets **after** the loss are thrown away and **sent again** → wasted bandwidth.
- Poor performance on **noisy networks**, where losses are frequent — a single loss costs a whole window.

GBN numericals from the slides

Ex 1. A sends 9 packets, window 3, every **5th transmission** is lost (ACKs never lost). How many transmissions?

```
1 2 3 4 5✗ 6 7 | 5 6 7✗ 8 9 | 7 8 9✗ | 9
7 + 5 + 3 + 1 = 16 transmissions
```

Ex 2. A sends 10 frames, Go-Back-4, every **6th** transmission corrupted.

```
1 2 3 4 5 6✗ 7 8 9 | 6 7 8✗ 9 10 | 8 9 10
9 + 5 + 3 = 17 transmissions
```

Method: number **every** transmission including retransmissions, mark each 5th/6th as lost, and after a loss resend from the lost

Selective Repeat — justify window ≤ 2^{m-1}

With **m-bit** sequence numbers there are **2^m** distinct numbers (0 … 2^m-1). In SR the **receiver buffers out-of-order packets inside its window**, so the sender window and receiver window must **never overlap** in sequence-number space: $W_{sender} + W_{receiver} \le 2^m$. With equal windows, **$W \le 2^{m-1}$**.

Example, m = 2 (numbers 0, 1, 2, 3):

- **W = 3 (too big):** sender sends 0, 1, 2. Receiver gets all three and slides its window to expect 3, 0, 1. **All ACKs are lost.** Sender times out and resends 0. The receiver is now expecting 0 *as a new packet*, so it accepts a **duplicate as new data** — an error.
- **W = 2 = 2^{2-1}:** after receiving 0, 1 the receiver's window is 2, 3. The resent 0 is **outside** its window, so it is recognised as a duplicate and simply re-ACKed. **Correct.**

Conclusion: a window larger than 2^{m-1} lets the receiver confuse an old retransmission with a new packet. *(In GBN the limit is $2^m - 1$ because the receiver window is only 1.)*

packet through the last one sent.

	GBN	Selective Repeat
Receiver buffer	**None**	**Buffers** out-of-order packets
ACKs	Cumulative	Individual per packet
Timers	One (oldest unACKed)	One **per packet**
On loss	Resend the whole window	Resend **only** the lost packet
Max window	$2^m - 1$	2^{m-1}

HOOK **GBN = "one timer, no buffer, cumulative ACK, resend everything."** SR = **"many timers, big buffer, individual ACK, resend one."** Window sizes: **G**BN gets the **G**reedier window (2^m-1); SR gets exactly half the space (2^{m-1}).

SCORING KEY Definition + "sender may send N without waiting" (1) · sender base/nextseqnum/one timer/cumulative ACK (1) · receiver discards and re-ACKs, no buffer (1) · the sequence diagram with the discard arrows and the go-back (2) · drawbacks (1).

14 RTT estimation, DevRTT and the TCP timeout interval — numerical

CAME IN CIE-1 APR 2025 AND CIE-1 MAY 2024 | 5–6 MARKS | CO2 | FREE MARKS

> *"Suppose the measured RTT value is 170 ms. (i) Compute EstimatedRTT using α = 0.125, assuming it was 150 ms before this sample. (ii) Compute DevRTT with β = 0.25, assuming it was 20 ms. (iii) Compute the TCP timeout interval."*
>
> CIE-1 Apr 2025 Q2b · CIE-1 May 2024 Q2c

$$\text{EstimatedRTT} = (1-\alpha)\cdot\text{EstimatedRTT} + \alpha\cdot\text{SampleRTT} \;=\; 0.875\cdot\text{old} + 0.125\cdot\text{Sample}$$

$$\text{DevRTT} = (1-\beta)\cdot\text{DevRTT} + \beta\cdot|\text{SampleRTT} - \text{EstimatedRTT}| \;=\; 0.75\cdot\text{old} + 0.25\cdot|\text{Sample} - \text{EstimatedRTT}|$$

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4\cdot\text{DevRTT}$$

Apr 2025 — one sample, 170 ms

```
EstimatedRTT = 0.875 × 150 + 0.125 × 170
             = 131.25 + 21.25 = 152.5 ms

DevRTT = 0.75 × 20 + 0.25 × |170 − 152.5|
       = 15 + 0.25 × 17.5
       = 15 + 4.375 = 19.375 ms

Timeout = 152.5 + 4 × 19.375
        = 152.5 + 77.5 = 230 ms
```

May 2024 — two samples, 106 ms then 120 ms (start 100 / 5)

```
— Sample 1 = 106 ms —
Est = 0.875×100 + 0.125×106 = 100.75 ms
Dev = 0.75×5 + 0.25×|106−100.75|
    = 3.75 + 1.3125 = 5.0625 ms
TO  = 100.75 + 4×5.0625 = 121 ms

— Sample 2 = 120 ms —
Est = 0.875×100.75 + 0.125×120 = 103.156 ms
Dev = 0.75×5.0625 + 0.25×|120−103.156|
    = 3.7969 + 4.2109 = 8.008 ms
TO  = 103.156 + 4×8.008 = 135.19 ms
```

TRAP · 3 MARKS LOST HERE EVERY YEAR (1) **Always compute EstimatedRTT first** — DevRTT uses the **new** EstimatedRTT, not the old one. (2) The **second sample uses the first sample's results** as its "old" values. (3) It is **4 × DevRTT**, never 2 or 3. (4) Show the substitution line, not just the answer — the marks are for the working.

HOOK "**Estimate, Deviate, then Wait.**" α = **1/8** = 0.125, β = **1/4** = 0.25, safety factor = **4**. So (1−α) = 0.875 and (1−β) = 0.75. **Why the formula exists:** SampleRTT fluctuates, so TCP keeps a smoothed average (EWMA) and adds a safety margin of 4 deviations so a normal fluctuation never triggers a spurious retransmission.

SCORING KEY All three formulas written before substituting (1.5) · correct EstimatedRTT (1) · correct DevRTT using the new EstimatedRTT (1.5) · correct timeout with units (1).

15 TCP connection management — 3-way handshake, closing, and the client states

ASKED 3× INCL. CIE-1 APR 2025 | 5–6 MARKS | CO2 | DRAW IT

> *"Analyze the significance of each step in the 3-way handshake and the connection termination process." / "With a neat diagram, identify the sequence of TCP states visited by a client TCP."*
>
> CIE-1 Apr 2025 Q3c · CIE-1 Jun 2023 Q2b · SEE Feb 2024 Q4c

A. Connection establishment — 3-way handshake

1. **SYN.** The client sends a segment with **SYN = 1** and a randomly chosen initial sequence number `client_isn`. No data.
   Significance: asks to open the connection and tells the server **where the client's byte numbering starts**.
2. **SYN + ACK.** The server replies with **SYN = 1**, its own `server_isn`, and `ack = client_isn + 1`; it **allocates buffers and variables**.
   Significance: agrees to connect, **confirms the client's number** and **announces its own**.
3. **ACK.** The client sends **SYN = 0**, `seq = client_isn + 1`, `ack = server_isn + 1`, and allocates its buffers. This segment **may already carry data**.
   Significance: confirms the server's number — the connection is now **ESTABLISHED**.

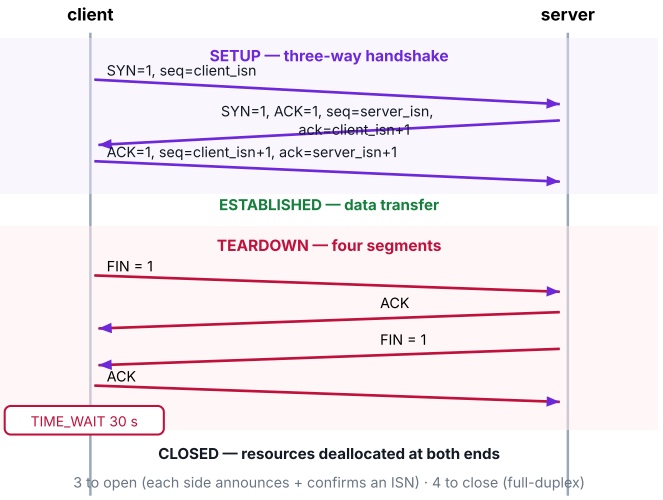
Why three steps: each side must **announce its own initial sequence number** and have it **acknowledged by the other**. Two steps cannot do both.

B. Connection termination — 4 segments

1. **FIN** from the client (no more data to send).
2. **ACK** from the server — the client→server direction is now closed.
3. **FIN** from the server (it has finished too).
4. **ACK** from the client. All resources in **both** hosts are deallocated; the client first waits in **TIME_WAIT** (~30 s).

Why four: TCP is **full-duplex**, so each direction must be shut down **separately**.

HOOK **"SYN · SYN-ACK · ACK"** to open, **"FIN · ACK · FIN · ACK"** to close. **3 to open, 4 to close** — because closing is one-direction-at-a-time. **TIME_WAIT** exists so a **lost final ACK can be re-sent** and so old duplicate segments die out before the port is reused.



Figs. 3.39 & 3.40 — opening and closing a TCP connection

C. The 6 client TCP states (Fig. 3.41)

#	State	Event / action → next
1	**CLOSED**	application calls `connect()`: **send SYN** → SYN_SENT
2	**SYN_SENT**	receive SYN & ACK, **send ACK** → ESTABLISHED
3	**ESTABLISHED**	data transfer here; application closes: **send FIN** → FIN_WAIT_1
4	**FIN_WAIT_1**	receive ACK, send nothing → FIN_WAIT_2
5	**FIN_WAIT_2**	receive FIN, **send ACK** → TIME_WAIT
6	**TIME_WAIT**	wait 30 seconds → CLOSED

First three = **connection setup**; last four = **connection termination**.

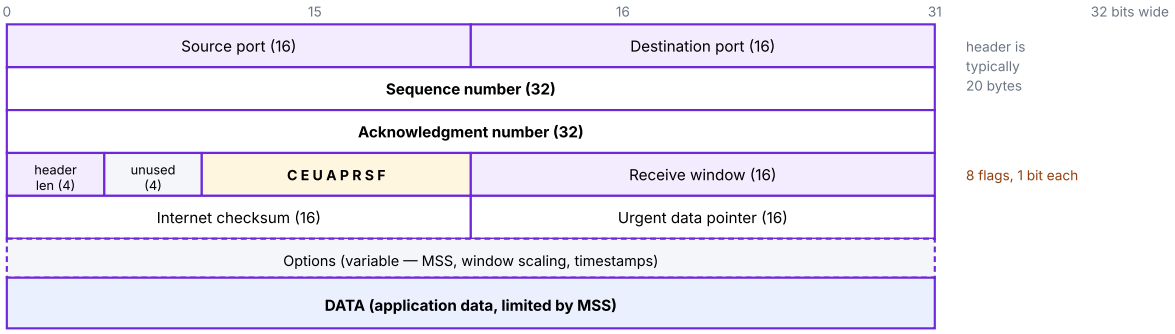
SCORING KEY The handshake diagram with seq/ack values (1.5) · the significance sentence for each of the 3 steps (1.5) · the 4-segment close + "why four: full-duplex" (1.5) · TIME_WAIT purpose (0.5). If the question says "states", draw the ring of 6 states instead and label every arrow event/action.

UNIT 2 Transport Layer

16 TCP segment structure and the 8 control flags

ASKED 3× INCL. CIE-1 APR 2025 & JUN 2023 | 5–7 MARKS | CO2 | DRAW IT

> *"Identify the various fields of the TCP segment structure." / "Identify the fields and explain any four flag fields."*
> **CIE-1 Apr 2025 Q3a · CIE-1 Jun 2023 Q1b · Makeup Feb 2020 Q3a**



C=CWR E=ECE U=URG A=ACK P=PSH R=RST S=SYN F=FIN
Header length 5–15 words → 20–60 bytes. MSS typically 1460 = MTU 1500 − 20 (IP) − 20 (TCP).

Fig. 3.29 — TCP segment structure. Draw the 32-bit-wide box exactly like this

Field	Size	Purpose
Source / destination port	16 each	Identify the sending & receiving **processes** (mux/demux)
Sequence number	32	**Byte-stream number of the first data byte** in this segment. TCP numbers **bytes**, not segments.
Acknowledgment number	32	**Next byte expected** from the other side. Valid when ACK = 1.
Header length	4	TCP header length in **32-bit words**
Unused / reserved	4	Set to 0
Flags	8	CWR ECE URG ACK PSH RST SYN FIN
Receive window	16	Bytes the receiver is willing to accept → **flow control**
Internet checksum	16	Error detection over **header and data**
Urgent data pointer	16	End of urgent data; valid only if URG = 1
Options	var.	MSS, window scaling, timestamps

Flag	Full form	When used
SYN	Synchronize	**Establish** a connection; synchronises the initial sequence numbers (step 1 of the handshake)
ACK	Acknowledgement	The ack-number field is **valid**; set in almost every segment after the connection is established
FIN	Finish	Sender has finished sending and wants an **orderly close**
RST	Reset	Error condition / **abort** the connection immediately
PSH	Push	Deliver the data to the application **at once**
URG	Urgent	Urgent pointer is valid
CWR	Congestion Window Reduced	Sender has cut its rate because of congestion
ECE	ECN Echo	Receiver tells the sender the network is congested

SEQ/ACK EXAMPLE Client sends "HELLO" with **Seq = 100**, so it carries bytes **100–104**. The server replies **ACK = 105**: "I have bytes 100–104; the next byte I expect is 105." **ACK = last byte received + 1.**

HOOK Flags in order **C E U A P R S F** → "**C**ute **E**lephants **U**sually **A**dmire **P**urple **R**oses, **S**unflowers, **F**igs." The **four to explain if asked for "any four"** are the four that actually do something visible: **SYN, ACK, FIN, RST**.

SCORING KEY The 32-bit-wide labelled box (2) · one line per field (2) · the four flags explained (1). For 7 marks add the seq/ack byte-numbering example.

17 Flow control and the receive window (rwnd)

ASKED 4× | 5–6 MARKS | CO2 | FORMULA

> *"With suitable formulas, explain what happens if the network layer delivers data faster than the application layer removes it from the socket buffers." / "Explain the flow control mechanism in TCP along with the receiver window calculations."*
>
> CIE-1 Jun 2023 Q3a · CIE-1 May 2024 Q3b · SEE Jan 2020 Q3b, Q3c

The problem: if data arrives faster than the application reads it, the **receive buffer fills up and overflows** — data is lost.

The solution: TCP **flow control** — a **speed-matching service** that matches the *sender's rate* to the *receiving application's reading rate*. *(Do not confuse with congestion control, which protects the **network**; flow control protects the **receiver**.)*

The receiver keeps `RcvBuffer` and two variables: **LastByteRead** (last byte the application read) and **LastByteRcvd** (last byte placed in the buffer). To avoid overflow:

$$\text{LastByteRcvd} - \text{LastByteRead} \le \text{RcvBuffer}$$

The free space is the **receive window**:

$$\text{rwnd} = \text{RcvBuffer} - [\ \text{LastByteRcvd} - \text{LastByteRead}\ ]$$

The receiver puts **rwnd in the receive-window field of every segment/ACK** it sends. Initially rwnd = RcvBuffer. The sender then limits its unacknowledged data:

$$\text{LastByteSent} - \text{LastByteAcked} \le \text{rwnd}$$

When rwnd = 0 the sender stops — but keeps sending **small probe segments** so it learns when space frees up. When the application reads data, a larger rwnd is advertised and sending resumes.

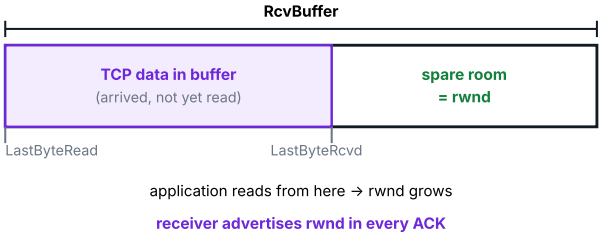


Fig. 3.38 — RcvBuffer = data in buffer + spare room (rwnd)

WORKED EXAMPLE RcvBuffer = **4096 B**, LastByteRcvd = **3000**, LastByteRead = **1000**.
rwnd = 4096 − (3000 − 1000) = **2096 B**. The sender may have at most **2096 bytes** unACKed.
If the application then reads up to byte 3000, rwnd rises back to **4096 B**.

THE 100 MBPS QUESTION A and B joined by a **100 Mbps** link; A's app writes at **120 Mbps**; B's app reads at only **50 Mbps**.
Data can arrive at B at up to **100 Mbps** (A's TCP cannot exceed the link) while B drains at **50**. So B's buffer fills, its ACKs advertise a **shrinking rwnd**, and eventually **rwnd = 0** → A stops (probing). **Long-run: A's average sending rate settles at ≈ 50 Mbps**, the link is only ~50 % utilised, and A's own send buffer fills so its application is throttled from 120 down to ≈ 50 Mbps too.

Buffering (if asked separately): the sending process writes to its socket; TCP places the data in the **send buffer**, cuts it into segments (data ≤ MSS) and sends them. The receiving TCP places arriving data in the **receive buffer** and the application reads from it at its own pace. Buffers exist **because the application and the network run at different speeds**.

HOOK **"rwnd protects the receiver, cwnd protects the network."** Data actually in flight = **min(cwnd, rwnd)**. And rwnd = **bucket size − water in it**.

SCORING KEY "speed-matching, prevents receive-buffer overflow" (1) · the rwnd formula and the sender's constraint (2) · the buffer diagram (1) · the rwnd = 0 probe rule (1).

18 Multiplexing and demultiplexing — connectionless and connection-oriented

ASKED 4× | 5–7 MARKS | CO2 | DRAW IT

> *"What is meant by multiplexing and demultiplexing? Explain with a neat diagram." / "Illustrate with an example connectionless demultiplexing." / "With a neat diagram explain connection-oriented multiplexing and demultiplexing."*
>
> SEE Jan 2020 Q4b · SEE Feb 2024 Q4b · CIE-1 Jun 2023 Q3c · Makeup Feb 2020 Q4b

- **Multiplexing (sender):** the transport layer gathers data from several application processes (through their **sockets**), adds a header with the **source and destination port numbers** to each chunk to form a **segment**, and passes the segments down to the network layer.
- **Demultiplexing (receiver):** the transport layer examines the **port-number fields** of each incoming segment, finds the **correct socket** and delivers the data to that process.
- **Why needed:** many applications (browser, chat, email, FTP) share **one host and one network connection**.

Connectionless demultiplexing — UDP

A UDP socket is identified by a **2-tuple: (destination IP, destination port)**. No handshake; every datagram is independent. The receiving host picks the socket using the **destination port alone**.

Example (Fig. 3.4): Host A's client process has socket port **19157** and sends a datagram to Server B's port **46428**. The segment carries source 19157, destination 46428. At B, UDP reads destination port 46428 and delivers to the process bound to it. B's reply **swaps the ports** — source 46428, destination 19157 — so A's UDP delivers it to the client process. If another host C also sends to port 46428, its datagram goes to the **same socket**; only the source IP/port differ, and they are used for the reply.



UDP uses the DESTINATION PORT only

Connection-oriented demultiplexing — TCP

A TCP socket is identified by a **4-tuple: (source IP, source port, destination IP, destination port)**. A connection is set up by a handshake first. When a segment arrives the host checks **all four values** and delivers it to the matching connection socket — so two segments with the **same destination port can still go to different sockets**.

Example (Fig. 3.5): web clients A and C connect to Web Server B on port 80.

```
A → B : (A, 26145, B, 80)
C → B : (C, 26145, B, 80)   same ports, different source IP
C → B : (C,  7532, B, 80)   different source port
```

The server therefore has **three different sockets**, each served by its own HTTP process, **even though all three use destination port 80**.

	UDP	TCP
Socket identified by	**2-tuple** (dest IP, dest port)	**4-tuple**
Same dest port from 2 hosts	Goes to the **same** socket	Goes to **different** sockets
Handshake	None	Yes

HOOK **"Mux = Many into one. Demux = one into Many."** And the counting rule: **UDP = 2, TCP = 4** — a UDP socket needs two numbers, a TCP socket needs four.

SCORING KEY Both definitions mentioning **sockets** and **port numbers** (1.5) · the 3-host / 5-layer diagram (1.5) · the 2-tuple vs 4-tuple rule (1) · one worked example with real port numbers (1).

19 UDP — why applications choose it, segment structure and checksum

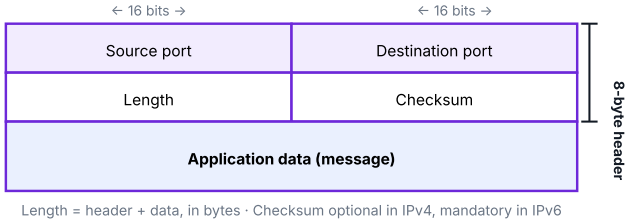
ASKED 3× INCL. CIE-1 MAY 2024 | 5–7 MARKS | CO2

> *"Why are some applications better suited for UDP than TCP? Justify." / "Explain the fields of the UDP segment structure with a neat diagram. With an example explain the UDP checksum."*
>
> CIE-1 May 2024 Q2b · Makeup Feb 2020 Q3b, Q4c

4 reasons to choose UDP

1. **Finer application-level control.** As soon as the application passes data, UDP wraps it in a segment and sends it. TCP may **delay** data (connection setup, congestion control, retransmission) — real-time apps cannot afford that.
2. **No connection establishment.** No three-way handshake, so **no setup delay** — this is exactly why **DNS uses UDP**.
3. **No connection state.** UDP keeps no buffers, no sequence/ack numbers, no congestion parameters, so a server can support **many more active clients**.
4. **Small header — 8 bytes vs TCP's 20 bytes.** Less overhead per packet.

Examples: internet telephony and video calls (delay-sensitive, tolerate a little loss), **DNS**, SNMP, NFS. Reliability can still be **built into the application** if needed.



UDP segment structure (RFC 768)

The 4 header fields

- **Source / destination port (16 bits each)** — identify the sending and receiving processes (multiplexing/demultiplexing).
- **Length (16 bits)** — total length of the UDP segment (header + data) in bytes.
- **Checksum (16 bits)** — error detection over the segment. Optional in IPv4, **mandatory in IPv6**.

UDP checksum — the standard example

Sender: 1's complement of the sum of all the 16-bit words, **wrapping any carry** back in.

```
  0110011001100000
+ 0101010101010101
  ----------------
  1011101110110101
+ 1000111100001100
  ----------------
  0100101011000010   (carry wrapped)

1's complement →
checksum = 1011010100111101
```

Receiver: adds **all four** 16-bit words *including* the checksum:

```
0100101011000010
+ 1011010100111101
  ----------------
  1111111111111111  → all 1s, NO ERROR
```

A **0 anywhere** in the result means the segment was **corrupted**.

HOOK UDP's four reasons = **"Control, Connection, Collection, Compact"** → finer app **control** · no **connection** setup · no connection state (so a bigger **collection** of clients) · **compact** 8-byte header. Header fields in order: **S-D-L-C** = Source, Destination, Length, Checksum.

SCORING KEY All 4 reasons, numbered (2) · the 8 vs 20 byte comparison (0.5) · named examples incl. DNS (0.5) · the 4-field diagram (1) · checksum sender + receiver working (2 if asked).

Unit 2 insurance — reliable data transfer in TCP

ASKED 2× IN SEES **READ ONCE**

The TCP sender has exactly 3 events. Variables: `NextSeqNum` (sequence number of the next byte to send) and `SendBase` (sequence number of the **oldest unacknowledged** byte).

```
NextSeqNum = InitialSeqNumber
SendBase   = InitialSeqNumber
loop (forever) {
  switch (event)
  event: data received from application above
      create TCP segment with sequence number NextSeqNum
      if (timer not running) start timer
      pass segment to IP
      NextSeqNum = NextSeqNum + length(data)
  event: timer timeout
      retransmit the not-yet-ACKed segment with the
        smallest sequence number
      start timer
  event: ACK received, with ACK field value y
      if (y > SendBase) {
          SendBase = y
          if (any not-yet-ACKed segments) start timer
      }
}
```

Explanation of the 3 events:

1. **Data from the application** — TCP makes a segment whose sequence number is the byte-stream number of its first byte, starts the timer (the timer always runs for the **oldest unACKed** segment, with interval `TimeoutInterval`), sends it to IP and advances `NextSeqNum` by the data length.
2. **Timeout** — the ACK did not arrive in time, so TCP assumes loss. It retransmits the unACKed segment with the **smallest** sequence number and restarts the timer (most implementations also **double** the interval).
3. **ACK received** — ACKs are **cumulative**. If y > SendBase then everything before y is acknowledged, so SendBase = y; if segments are still unACKed the timer is restarted.

THE CLASSIC SUM B has received all bytes through **126**. A sends two segments back-to-back: **80 bytes with Seq = 127** (source port 302, dest 80) and **40 bytes with Seq = 207**.
(i) First arrives first → B has bytes through 206, so **ACK = 207**, source port **80**, destination port **302** (ports swap in the reply).
(ii) Second arrives first → B still hasn't got byte 127, so **ACK = 127**.
(iii) If the first ACK is lost and the second arrives after A's timeout: A retransmits Seq = 127; B discards the duplicate and sends **ACK = 247**. Because ACKs are **cumulative**, ACK = 247 tells A all bytes up to 246 arrived, so **only segment 1 was retransmitted**.

Always write: ACK number = **next expected byte = last byte received + 1**.

UNIT 3 Network Layer — up to IPv4 Addressing

8 questions · CO3 · Forouzan 7.4 · 80% numerical

READ FIRST — THE MASTER METHOD Every Unit 3 numerical is the same four lines. Learn these and you can answer **any** addressing question in the paper:
1. $N = 2^{32-n}$ · **2.** mask = n ones then (32−n) zeros · **3.** first address = **address AND mask** (keep the n leftmost bits, zero the rest) · **4.** last address = **address OR NOT(mask)** (keep the n leftmost bits, set the rest to 1). Also: N = NOT(mask) + 1, and $n = 32 - \log_2 N$.

20 Given a CIDR address, find the mask, N, first and last address

CAME IN CIE-1 APR 2025 · GUARANTEED MARKS | 4–5 MARKS | CO3 | NUMERICAL

> *"An IP address is given in CIDR format: 192.168.10.55/26. (i) Calculate the total number of valid IP addresses. (ii) Determine the first IP address (network address)." / "A classless IPv4 address is 156.180.150.89/26. Calculate the address mask and use it to find the first address, last address and range."*
>
> CIE-1 Apr 2025 Q1c · SEE Jan 2020 Q5b

Worked — 192.168.10.55/26 (the Apr 2025 question)

```
(i) Number of addresses
host bits = 32 − 26 = 6
N = 2⁶ = 64 addresses in the block
usable hosts = 64 − 2 = 62
  (network address and broadcast excluded)

(ii) Mask and first address
/26 → 11111111.11111111.11111111.11000000
     → 255.255.255.192

last octet 55 = 00110111
keep first 2 bits, zero the rest:
     00|110111  →  00|000000 = 0
first (network) address = 192.168.10.0

last address: set the 6 host bits to 1
     00|111111 = 63
last (broadcast) address = 192.168.10.63
range = 192.168.10.0 – 192.168.10.63
```

TRAP "Total number of **valid** IP addresses" is deliberately ambiguous. **Write both:** "**64** addresses in the block, of which **62** are usable host addresses." You cannot lose a mark that way.

Worked — 156.180.150.89/26 by the mask method

```
mask = 255.255.255.192
NOT(mask) = 0.0.0.63

N = NOT(mask) + 1 = 63 + 1 = 64    (= 2³²⁻²⁶)

first = address AND mask
  89  = 01011001
  192 = 11000000
  AND = 01000000 = 64
  → 156.180.150.64

last = address OR NOT(mask)
  89  = 01011001
  63  = 00111111
  OR  = 01111111 = 127
  → 156.180.150.127

range = 156.180.150.64 – 156.180.150.127
```

When only the *class* is given — e.g. 73.22.17.25

- First octet **73** is in 0–127 → **Class A**, so n = **8**.
- $N = 2^{32-8} = 2^{24}$ = **16,777,216**.
- First: keep 8 bits, rest 0 → **73.0.0.0**. Last: rest 1 → **73.255.255.255**.

The mask table — memorise the last-octet column

Prefix	/24	/25	/26	/27	/28	/29	/30
Last octet of mask	0	**128**	**192**	**224**	**240**	248	252
Addresses (N)	256	128	64	32	16	8	4
Usable hosts (N−2)	254	126	62	30	14	6	2
Subnets from a /24	1	2	4	8	16	32	64

HOOK Mask last octet goes **128 → 192 → 224 → 240 → 248 → 252** — each step **adds the next halved number** (128, +64, +32, +16, +8, +4). Addresses halve the other way: **256 → 128 → 64 → 32 → 16 → 8 → 4**. And **"AND for first, OR-NOT for last, NOT+1 for the count."**

SCORING KEY Mask written in binary *and* dotted decimal (1) · N with the 2^{32-n} working (1) · first address with the AND shown in binary (1) · last address with the OR (1) · the range written out (0.5).

21 Subnetting design — unequal sub-blocks from one granted block

MOST-ASKED UNIT 3 TOPIC (5×) | 6–9 MARKS | CO3 | NUMERICAL

> *"An organization is granted a block with the beginning address 14.24.74.0/24. It needs three subblocks: one of 10 addresses, one of 60 and one of 120. Design the sub-blocks."*
>
> SEE Feb 2024 Q5b (9 m) · CIE-2 Jul 2024 Q1a (6 m)

Step 1 — the given block

n = 24 → N = 2^{32-24} = 2^8 = **256 addresses**: **14.24.74.0 to 14.24.74.255**.

Step 2 — the 3 design rules (write these, they carry marks)

1. The number of addresses in each subnet must be a **power of 2** — round every requirement **up**.
2. The prefix length of each subnet is **$n_{sub} = 32 - \log_2 N_{sub}$**.
3. The **starting address of each subnet must be divisible by its own size**. This works out automatically if you **allocate the largest subnet first**.

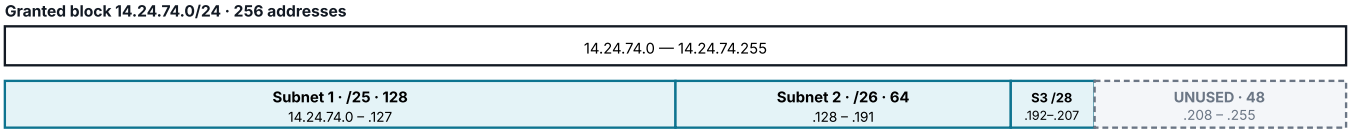
Step 3 — allocation (largest first: 120 → 60 → 10)

Subnet	Need → size	Prefix	Mask	First address	Last address
1	120 → **128** (2^7)	32 − 7 = **/25**	255.255.255.128	14.24.74.0	14.24.74.127
2	60 → **64** (2^6)	32 − 6 = **/26**	255.255.255.192	14.24.74.128	14.24.74.191
3	10 → **16** (2^4)	32 − 4 = **/28**	255.255.255.240	14.24.74.192	14.24.74.207

Last address = first address + N_{sub} − 1. *(128 + 64 − 1 = 191, 192 + 16 − 1 = 207.)*

Step 4 — leftover

Used = 128 + 64 + 16 = **208**. Unused = 256 − 208 = **48 addresses: 14.24.74.208 – 14.24.74.255**.



Draw this bar (Fig. 7.10). Note the widths are proportional: 128 : 64 : 16 : 48.

Fig. 7.10 — the original block, the three sub-blocks and the unused part

HOOK **"Biggest first, or the maths breaks."** If you allocated the 16-block first it would start at .0 and the 128-block could not then start on a multiple of 128. Round-up table to have in your head: **10→16, 50→64, 60→64, 100→128, 120→128, 250→256, 500→512, 1000→1024**.

SCORING KEY Step 1 with N = 2^{32-n} (1) · the 3 rules stated (1.5) · the allocation table with prefix + first + last for each (3) · masks (1) · unused addresses computed (1) · the block diagram (1.5).

UNIT 3 Network Layer — up to IPv4 Addressing

continued · Q22 – Q25

22 ISP block distributed among many organisations / customer groups

ASKED 2× IN SEES · 6–8 MARKS · CO3 · NUMERICAL

> *"An ISP is granted the block 80.70.56.0/21. It must allocate for two organisations of 500 addresses, two of 250 and three of 50. (i) Find the number and range of addresses in the ISP block. (ii) Find the range for each organisation."*
>
> SEE Jan 2020 Q5c · Makeup Feb 2020 Q5a

(i) The ISP block

/21 → N = $2^{32-21} = 2^{11}$ = **2048 addresses**.
Range: **80.70.56.0 to 80.70.63.255** — the third octet runs **56 to 63** (2048 ÷ 256 = 8 full /24s).

(ii) Allocation, largest first

Org	Need → size	Block	Range
1	500 → **512**	80.70.56.0/**23**	80.70.56.0 – 80.70.57.255
2	500 → **512**	80.70.58.0/**23**	80.70.58.0 – 80.70.59.255
3	250 → **256**	80.70.60.0/**24**	80.70.60.0 – 80.70.60.255
4	250 → **256**	80.70.61.0/**24**	80.70.61.0 – 80.70.61.255
5	50 → **64**	80.70.62.0/**26**	80.70.62.0 – 80.70.62.63
6	50 → **64**	80.70.62.64/**26**	80.70.62.64 – 80.70.62.127
7	50 → **64**	80.70.62.128/**26**	80.70.62.128 – 80.70.62.191

Check (always write it): used = 512×2 + 256×2 + 64×3 = **1728**; unused = 2048 − 1728 = **320** addresses (80.70.62.192 – 80.70.63.255).

The "groups of customers" variant

"An ISP is granted 190.100.0.0/16 (65,536 addresses). Group 1 = 64 customers × 256 addresses, Group 2 = 128 customers × 128, Group 3 = 128 customers × 64. Design the sub-blocks and find what is left."

Grp	Customers × size	Total	Each gets	Range of the group
1	64 × 256	16,384	**/24** 190.100.0.0/24 … 190.100.63.0/24	190.100.0.0 – 190.100.63.255
2	128 × 128	16,384	**/25** 190.100.64.0/25 … 190.100.127.128/25	190.100.64.0 – 190.100.127.255
3	128 × 64	8,192	**/26** 190.100.128.0/26 … 190.100.159.192/26	190.100.128.0 – 190.100.159.255

Allocated = 16,384 + 16,384 + 8,192 = **40,960**.
Still available = 65,536 − 40,960 = **24,576** addresses: **190.100.160.0 to 190.100.255.255**.

Prefix check: 32 − $\log_2 256$ = 24 · 32 − $\log_2 128$ = 25 · 32 − $\log_2 64$ = 26.
Each group's range runs from the **first customer's first address** to the **last customer's last address**.

HOOK Third-octet arithmetic: **512 addresses = 2 whole /24s**, so a /23 advances the third octet by 2. **256 = 1 third-octet step**, **64 = quarter of the last octet**. Convert "how many addresses" into "how many third-octet steps" and the ranges write themselves.

SCORING KEY N and the full ISP range (1) · the allocation table, largest first, with prefix + range per organisation (4) · the used/unused check (1) · a one-line statement of the three rules (1).

23 Splitting a block into equal-sized subnets

ASKED IN SEE JAN 2020 | 5–6 MARKS | CO3 | NUMERICAL

> *"An ISP owns the block 128.119.40.64/26. It wants to create four subnets from this block, each with the same number of IP addresses. What are the ranges (first and last address) of the four subnets?"*
>
> SEE Jan 2020 Q6c

```
Step 1  /26 → N = 2^(32−26) = 2^6 = 64 addresses
        block = 128.119.40.64 – 128.119.40.127

Step 2  4 equal subnets → 64 / 4 = 16 each
        16 = 2^4 → borrow 2 extra bits
        new prefix = 26 + 2 = /28
        mask = 255.255.255.240

Step 3  each subnet spans 16 addresses, so the
        first addresses step by 16:
        64, 80, 96, 112
```

HOOK **Each borrowed bit doubles the subnets and halves their size.** Need 4 subnets → 4 = 2^2 → borrow **2** bits. Need 8 → borrow 3. The prefix always becomes **old n + borrowed bits**.

Subnet	Network	First address	Last address
1	128.119.40.64/28	128.119.40.64	128.119.40.79
2	128.119.40.80/28	128.119.40.80	128.119.40.95
3	128.119.40.96/28	128.119.40.96	128.119.40.111
4	128.119.40.112/28	128.119.40.112	128.119.40.127

Each subnet has **16 − 2 = 14 usable hosts** (the first address is the **network address**, the last is the **broadcast address**).

Borrowed-bits table (from a /24) — memorise

Borrowed bits	1	2	3	4	5	6
Last-octet mask	128	192	224	240	248	252
Subnets	2	4	8	16	32	64
CIDR	/25	/26	/27	/28	/29	/30
Addresses each	128	64	32	16	8	4
Valid hosts	126	62	30	**14**	6	2

TRAP Some slide copies print "16 valid hosts" for /28 — that is a typo. **Valid hosts = addresses − 2 = 14.**

SCORING KEY N of the original block (1) · dividing to get the subnet size and the borrowed bits (1) · new prefix **and** mask (1) · the 4-row table of first/last addresses (2) · usable-hosts line (0.5).

24 Classful addressing with a block-allocation diagram, and the need for CIDR

ASKED 2× | 5–6 MARKS | CO3 | DRAW IT

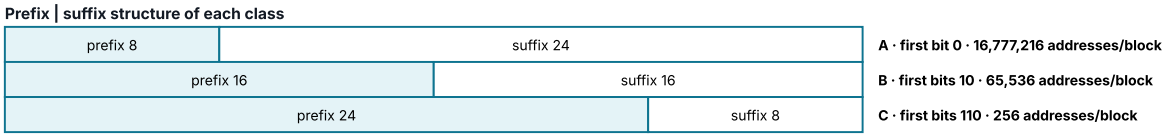
> *"Illustrate classful addressing with a block allocation diagram. What is the need for CIDR?"*
> SEE Feb 2024 Q5a · CIE-2 Jul 2024 Q2a

1. Definition: in classful addressing the whole address space (2^{32} **= 4,294,967,296** addresses) is divided into **5 fixed classes, A to E**. **The class fixes the size of the network and host parts**; an organisation is allocated a whole block of its class and cannot get a smaller or larger one.

2. Block-allocation diagram — share of the whole address space



Prefix | suffix structure of each class



Class	First bits	First byte	Prefix | suffix	Block size	# blocks	Default mask	Given to
A	0	0–127	8 | 24	2^{24} = 16,777,216	128	255.0.0.0 (/8)	very large organisations
B	10	128–191	16 | 16	2^{16} = 65,536	16,384	255.255.0.0 (/16)	medium organisations
C	110	192–223	24 | 8	2^{8} = 256	2,097,152	255.255.255.0 (/24)	small organisations
D	1110	224–239	No prefix/suffix split — **multicast**				
E	1111	240–255	No prefix/suffix split — **reserved for future use**				

Usable hosts per network = $2^{\text{suffix}} - 2$ → A 16,777,214 · B 65,534 · C 254 (network and broadcast excluded).
How the first-byte range is derived: take the fixed bits, count the k variable bits, range = fixed value to fixed + $2^{k} - 1$. Class B: fixed `10` = 128, 6 variable bits → 128 to 128 + 64 − 1 = **191**.

3. The need for CIDR (classless addressing)

- **Fixed sizes waste addresses.** A company needing **50** addresses must take a whole class C block of **256** → **204 wasted**. One needing **1000** must take a class B of **65,536** → almost all wasted.
- This caused **address depletion** — IPv4 has only ≈ **4.3 billion** addresses while computers, phones and IoT devices keep joining.
- **CIDR allocates blocks of any size that is a power of 2**, using a prefix **/n**, so each organisation gets what it actually needs. It also enables **address aggregation** (many small blocks advertised as one route).
- **Long-term solution:** IPv6 (larger address space, new packet format). **Short-term solution:** classless addressing.

HOOK First bits grow by one per class: **0 · 10 · 110 · 1110 · 1111**. First-byte boundaries: **127 / 191 / 223 / 239** — remember as **"128, 192, 224, 240 minus one"**. Share of space halves each time: **50 · 25 · 12.5 · 6.25 · 6.25**.

SCORING KEY Definition + 2^{32} figure (1) · the percentage bar + prefix/suffix picture (1.5) · the A–E table with first-byte ranges (1.5) · the CIDR-need list with the 50-address example (2).

25 Classful vs classless addressing — and why classless is more efficient

CAME IN CIE-1 APR 2025 | 5 MARKS | CO3

> *"Compare classful and classless addressing. Why is classless addressing more efficient for modern IP allocation?"*
> CIE-1 Apr 2025 Q1c (first half)

	Classful	Classless (CIDR)
Network size	**Fixed** — class A, B or C only	**Variable**, chosen to fit the requirement
Notation	No slash notation; the class is implied by the first bits	**CIDR /n** notation, e.g. 192.168.1.0/26
Block sizes	Only 256 / 65,536 / 16,777,216	**Any power of 2**
Wastage	Large — cannot create smaller or larger networks	Small — reduces wastage
Routing	No aggregation	Supports **address aggregation** (route summarisation)
Need 50 addresses?	Class C /24 → 256 total, 254 usable, **204 wasted**	/26 → 64 total, 62 usable, **only 12 spare**

Why classless is more efficient (the marks sentence)

The block matches the requirement. A company needing 50 addresses receives a **/26** (64 addresses, 62 usable) instead of a whole class C (256, wasting 204). Because every allocation is sized to need:

- the **4.3-billion address space lasts far longer** (slows address depletion);
- ISPs can **aggregate** many customer blocks into a **single routing entry**, which keeps routing tables small;
- allocation is done by **ICANN** to ISPs in large blocks, which the ISP then subdivides freely.

TWO RESTRICTIONS ON ANY ALLOCATED BLOCK (1) the number of addresses **N must be a power of 2**, because $n = 32 - \log_2 N$ must be an integer; (2) the addresses must be **contiguous**, and the **first address must be divisible by the block size**.
Slide example: an ISP requests **1000** addresses — not a power of 2, so **1024** (2^{10}) are granted and $n = 32 - 10 =$ **/22**.

HOOK **"Classful gives you a T-shirt in S, M or L. CIDR is a tailor."** The one number to quote in every CIDR answer: **50 addresses → /26 → 64 → 62 usable**, versus **class C → 256 → 204 wasted**.

SCORING KEY The comparison table with ≥4 rows (2) · the efficiency explanation with the 50-address worked comparison (2) · address aggregation / depletion mentioned (1).

UNIT 3 Network Layer — up to IPv4 Addressing

continued · Q26 – Q27 + insurance

26 Network layer — role, 4 functions and the services it provides

CIE-1 JUN 2023 Q2A · 5 MARKS · CO3 + CO1

> *"Show the layers in the TCP/IP protocol suite and explain the services provided by the network layer."*
> **CIE-1 Jun 2023 Q2a — note this question bridges Unit 1 and Unit 3**

1. The stack (one line each)

- **Application** — HTTP, SMTP, DNS, FTP · object = **message**
- **Transport** — TCP / UDP · object = **segment / user datagram** · address = **port**
- **Network** — IP · object = **datagram / packet** · address = **IP address**
- **Data link** — Ethernet, Wi-Fi · object = **frame** · address = **MAC**
- **Physical** — bits on the medium

2. Role of the network layer

It is the **3rd layer of the OSI model**. Its main job is to **deliver packets from the source device to the destination device even when they are on different networks**. Its most important protocol is **IP**, which provides logical addressing, packet delivery across networks and routing support.

3. The 4 main functions / services

Function	Meaning
Logical addressing	Gives every device an **IP address** so it can be identified
Routing	**Finding a path** from source to destination
Packet forwarding	When a router receives a packet it decides **where to send it next** (next hop)
Internetworking	Lets **different networks** communicate with each other

Always add: IPv4 is **connectionless** and **best-effort** — it sends each packet independently and does not guarantee delivery or ordering (that is TCP's job).

Network-layer protocols

IPv4 addressing & delivery · **IPv6** next-gen addressing · **ICMP** error reporting & diagnostics · **IPsec** security · **OSPF, RIP** routing · **BGP** routing between AS

HOOK Four functions = **"Locate, Route, Forward, Interconnect"** (L-R-F-I). The one distinction examiners test: **routing = choosing the path (done in advance, builds the table); forwarding = moving one received packet to its next hop (done per packet).**

SCORING KEY The 5-layer stack with one line each (2) · the 4 network-layer services named and explained (2) · connectionless/best-effort line (1).

27 IPv4 address basics — 32 bits, 3 notations, prefix/suffix, finding the class

FOUNDATION FOR EVERY UNIT 3 NUMERICAL | 4–5 MARKS | CO3

> *Short-answer / objective-style parts drawn from the slide exercises — these appear as sub-parts of larger addressing questions.*
>
> Slides p. 8–18, 52–55

The address

- An IPv4 address is **32 bits**; each 8-bit part is an **octet**.
- Written as four decimals separated by dots, each **0 to 255** ($2^8 - 1$).
- **Address space** = total addresses a protocol can use = 2^b. IPv4: **2^{32} = 4,294,967,296**.

3 notations

Decimal	Binary	Hex
128	10000000	80
11	00001011	0B
3	00000011	03
31	00011111	1F

128.11.3.31 = `10000000 00001011 00000011 00011111` = **800B031F**

Hierarchy: prefix & suffix

Prefix = the **network** part, the first **n** bits — tells a router *which network* the device is on. **Suffix** = the **host** part, the remaining **32 − n** bits — identifies the device inside that network.

Address	Prefix n	Host bits	Addresses
192.168.1.25/24	24	8	2^8 = 256
172.16.10.25/16	16	16	2^{16} = 65,536

Finding the class

Rule: look at the first byte (or the first bits).

Address	First byte/bits	Class
192.68.1.18	192 (192–223)	**C**
48.8.1.1	48 (0–127)	**A**
250.16.1.1	250 (240–255)	**E**
122.1.1.1	122 (0–127)	**A**
00000001 …	first bit 0	**A**
11000001 …	110	**C**
10100111 …	10	**B**
11110011 …	1111	**E**

"Spot the error" — slide exercise

Address	Why it is invalid
111.56.**045**.78	**No leading zero** is allowed
221.34.7.8.**20**	IPv4 has **only 4 octets**
75.45.**301**.14	Each octet must be **0–255**
11100010.23.14.67	Cannot **mix binary and dotted-decimal**

Why the network address matters

The **network address** (first address of a block) is what a router uses to **route a packet to its destination network**. A router with *m* interfaces, on receiving a packet, finds the **network address of the destination**, looks it up in its **forwarding table** and sends the packet out of the matching interface.

Address aggregation

Several contiguous small blocks given to one ISP can be advertised to the outside world as **one larger block / one routing entry** (route summarisation). This is only possible with **classless** addressing and it keeps global routing tables small.

HOOK **ICANN** gives big blocks to **ISPs**, never to individual users. **Two restrictions:** N must be a power of 2, and the addresses must be contiguous starting at an address divisible by N.

SCORING KEY 32 bits / 4 octets / 2^{32} (1) · the 3 notations with one conversion worked (1.5) · prefix vs suffix (1) · class identification rule with examples (1.5).

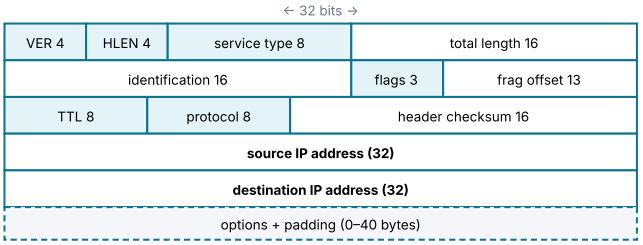
+ Beyond "IPv4 Addressing" — 5-minute insurance

OUTSIDE YOUR STATED PORTION | BUT INSIDE IPV4 AND ASKED 4× IN SEES/CIE-2 | **SKIM ONLY**

Your portion stops at IPv4 *addressing*. The **IPv4 datagram format** and **fragmentation** are slides 89–143 of the same IPv4 section, so a question *could* stretch there. Two minutes each — read, don't memorise.

IPv4 datagram format

Datagram = **header (20–60 bytes) + payload**, total 20–65,535 bytes. IPv4 is **connectionless with no delivery guarantee**.



- **VER** (4) = 4 · **HLEN** (4) = header length in 4-byte words, 5–15 → **20–60 bytes**.
- **Service type** (8) — QoS: precedence, delay, throughput, reliability.
- **Total length** (16) — header + data in bytes; **data = total length − HLEN × 4**.
- **Identification** (16) — packet ID, **same for all fragments** of a datagram.
- **Flags** (3) — unused, **DF** (don't fragment), **MF** (more fragments).
- **Fragmentation offset** (13) — position of the fragment's data, in **8-byte units**.
- **TTL** (8) — hop limit, stops datagrams looping.
- **Protocol** (8) — **ICMP 1, TCP 6, UDP 17**.
- **Header checksum** (16) — checks the **header only**; a bad header is discarded.
- **Source / destination address** (32 each) · **Options** (0–40 bytes).

Fragmentation

Definition: dividing an IP datagram into smaller datagrams (**fragments**) so each fits in the frame of the next network.

Need: a datagram passes through different physical networks, each with its own **MTU** (maximum transfer unit — the largest frame payload it can carry): **Ethernet 1500, FDDI 4352, PPP 512**. When the datagram is bigger than the next network's MTU it cannot be sent as it is, so the router (or source host) splits it.

Rules: the fragmenting device changes **flags, fragmentation offset and total length** (other fields are copied, the checksum is recalculated). All fragments carry the **same identification**; offset is in **8-byte units**; **MF = 1** on all but the last. **Only the final destination reassembles.**

THE STANDARD SUM Packet **6000 bytes**, MTU **1500**, IP header 20 bytes (+ another 20-byte attached header).
IP payload to split = 6000 − 20 = **5980 bytes**.
Max data per fragment = 1500 − 20 = **1480** (a multiple of 8 ✓).
5980 ÷ 1480 = 4.04 → **5 fragments**.

Frag	Total len	Data	ID	MF	Offset
1	1500	1480	x	1	0
2	1500	1480	x	1	1480/8 = 185
3	1500	1480	x	1	2960/8 = 370
4	1500	1480	x	1	4440/8 = 555
5	80	60	x	**0**	5920/8 = 740

HOOK **Offset is always in units of 8 bytes** — divide the byte position by 8. **MF = 1 on every fragment except the last.** ID is identical on all of them.

VAULT Every mnemonic on one page

read this in the corridor

Unit 1

OSI 7 LAYERS Top→bottom: **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing = Application, Presentation, Session, Transport, Network, Data link, Physical. Bottom→top: **P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way.

OBJECTS PER LAYER **M**y **S**ister **D**raws **F**unny **B**irds = Message, Segment, Datagram, Frame, Bits.

LAYERS PER DEVICE **Host 5 · Router 3 · Switch 2.**

DNS RECORD TYPES **"A NaMe Can Mail"** = A, NS, CNAME, MX. Tuple order **N-V-T-T** = Name, Value, Type, TTL.

DNS SERVER CLASSES **R**oot **T**ells, **T**LD **T**ells, **A**uthoritative **A**nswers. Quote: **13** root servers, **12** organisations, **IANA**.

COOKIE COMPONENTS **Set it, Send it, Store it, Search it** = Set-cookie header, Cookie header, browser cookie file, back-end database.

SMTP COMMANDS **H-M-R-D-Q**: **H**ELO, **M**AIL FROM, **R**CPT TO, **D**ATA, **Q**UIT.

TOPOLOGIES **"My Star Bus Rings."** Only **Bus is multipoint**; the other three are point-to-point. Mesh links = **n(n−1)/2**.

DATA COMMUNICATION 4 characteristics: **"Don't Arrive Too Jittery"** = Delivery, Accuracy, Timeliness, Jitter.
5 components: **"Most Students Read The Paper"** = Message, Sender, Receiver, Transmission medium, Protocol.

Unit 2

CONGESTION CONTROL MECHANISMS **S-A-R-R** = **S**tart slow, **A**void trouble, **R**esend fast, **R**ecover. (Slow start, Congestion avoidance, Fast retransmit, Fast recovery.)

TCP FLAGS IN ORDER **C E U A P R S F** = "**C**ute **E**lephants **U**sually **A**dmire **P**urple **R**oses, **S**unflowers, **F**igs."

UDP HEADER FIELDS **S-D-L-C** = Source port, Destination port, Length, Checksum. **8 bytes.**

WHY UDP **Control, Connection, Collection, Compact** = finer app control · no connection setup · no connection state (more clients) · compact 8-byte header.

SOCKET TUPLES **UDP = 2, TCP = 4.** UDP socket = (dest IP, dest port). TCP socket = (src IP, src port, dest IP, dest port).

GBN VS SR GBN = **one timer, no buffer, cumulative ACK, resend everything**. SR = **many timers, buffer, individual ACK, resend one**. Windows: GBN ≤ 2^m-1, SR ≤ 2^{m-1}.

CONNECTION MANAGEMENT **3 to open** (SYN · SYN-ACK · ACK), **4 to close** (FIN · ACK · FIN · ACK) — because TCP is full-duplex and each direction closes separately.

THE TWO WINDOWS **rwnd protects the receiver · cwnd protects the network.** Data in flight = **min(cwnd, rwnd)**.

RTT CONSTANTS α = **1/8** (0.125) · β = **1/4** (0.25) · safety factor **4**. So you multiply by **0.875** and **0.75**.

Unit 3

THE THREE MASK FORMULAS **"AND for first, OR-NOT for last, NOT+1 for the count."**
first = address AND mask · last = address OR NOT(mask) · N = NOT(mask) + 1 = 2^{32-n}.

MASK LAST OCTET **128 · 192 · 224 · 240 · 248 · 252** for /25 /26 /27 /28 /29 /30.
Addresses: **128 · 64 · 32 · 16 · 8 · 4**.
Usable: subtract 2.

CLASS FIRST BITS **0 · 10 · 110 · 1110 · 1111** — one more bit each class.
First-byte boundaries: **127 / 191 / 223 / 239 / 255**.

ADDRESS SPACE SHARE A **50%** · B **25%** · C **12.5%** · D **6.25%** · E **6.25%** — halves each time.

SUBNET DESIGN **"Biggest first, or the maths breaks."** Round up to a power of 2 → $n_{sub} = 32 - \log_2 N_{sub}$ → next block starts right after the previous last address.

NETWORK LAYER FUNCTIONS **"Locate, Route, Forward, Interconnect"** = logical addressing, routing, packet forwarding, internetworking.

ROUTING VS FORWARDING **Routing chooses the path** (builds the table in advance). **Forwarding moves one packet** to its next hop.

ROUND-UP TABLE 10→16 · 50→64 · 60→64 · 100→128 · 120→128 · 250→256 · 500→512 · 1000→1024.

FRAGMENTATION **Offset is in 8-byte units. MF = 1** on every fragment but the last. **ID identical** on all.

LAST 10 MIN Formula & number card

if you read nothing else, read this

Formulas you will actually substitute into

$$EstimatedRTT = 0.875 \cdot old + 0.125 \cdot Sample$$

$$DevRTT = 0.75 \cdot old + 0.25 \cdot |Sample - \underline{new}\ EstimatedRTT|$$

$$TimeoutInterval = EstimatedRTT + 4 \cdot DevRTT$$

$$rwnd = RcvBuffer - [LastByteRcvd - LastByteRead]$$

$$LastByteSent - LastByteAcked \le rwnd \quad \cdot \quad in\ flight = \min(cwnd, rwnd)$$

$$Fast\ recovery:\ ssthresh = cwnd/2 \ , \ cwnd = ssthresh + 3\ MSS$$

$$Timeout:\ ssthresh = cwnd/2 \ , \ cwnd = 1\ MSS$$

$$N = 2^{32-n} \quad \cdot \quad n = 32 - \log_2 N \quad \cdot \quad N = NOT(mask) + 1$$

$$first = address\ AND\ mask \quad \cdot \quad last = address\ OR\ NOT(mask)$$

$$last = first + N_{sub} - 1 \quad \cdot \quad usable\ hosts = N - 2$$

$$Mesh\ links = n(n-1)/2 \quad \cdot \quad ports\ per\ device = n - 1$$

$$Non\text{-}persistent\ HTTP = 2\ RTT + transmit,\ per\ object$$

$$D_{cs} = \max \{ NF/u_s \ , \ F/d_{min} \}$$

$$D_{P2P} = \max \{ F/u_s \ , \ F/d_{min} \ , \ NF/(u_s + \Sigma u_i) \}$$

Numbers that earn marks when quoted

Thing	Number
Port: HTTP / HTTPS	**80 / 443**
Port: SMTP · POP3 · IMAP	**25 · 110 · 143**
Port: FTP control / data	**21 / 20**
Port: DNS	**53** (over UDP)
UDP header / TCP header	**8 bytes / 20 bytes**
TCP header range	20–60 bytes (HLEN 5–15 words)
MSS / MTU	**1460 / 1500** (1500 − 20 − 20)
Port number size	16 bits
IP address / MAC address	**32 bits / 48 bits**
IPv4 address space	2^{32} = **4,294,967,296**
IPv4 header	20–60 bytes; datagram ≤ 65,535
Protocol numbers	ICMP **1**, TCP **6**, UDP **17**
MTUs	Ethernet **1500**, FDDI 4352, PPP 512
Root DNS servers	**13** (1000+ instances, 12 orgs, IANA)
TIME_WAIT	**30 s**
Initial ssthresh (FSM)	**64 KB**; cwnd = 1 MSS
Duplicate ACKs for fast retransmit	**3**
Class A/B/C block sizes	16,777,216 / 65,536 / 256
Class A/B/C blocks available	128 / 16,384 / 2,097,152

THE FIVE MOST COMMON MARK-LOSERS

1. Using the **old** EstimatedRTT inside DevRTT. Compute EstimatedRTT first, always.
2. Writing only "64 addresses" when the question says **valid** — write **64 in the block, 62 usable**.
3. Allocating the **smallest** subnet first, so later blocks don't start on a multiple of their size.
4. Describing GBN but drawing no **discard + re-ACK** arrows — that is where the marks are.
5. Writing prose with **no diagram** for an "Illustrate/Draw/Show" verb. Those verbs are worth 2 marks of picture.

Final 60 seconds — the three sentences to carry in

Unit 1. Every application-layer answer is **client + server + protocol + port + a numbered message exchange**. If you can name the two endpoints, the port and the order of messages, you have the marks.

Unit 2. Everything is one of two windows. **rwnd** = what the receiver can take. **cwnd** = what the network can take. Loss is detected by **timeout** or **3 duplicate ACKs**, and those two events have different recoveries.

Unit 3. Four lines solve everything: **$N = 2^{32-n}$**, mask = n ones, **first = AND**, **last = OR NOT**. Round up to a power of two, allocate largest first, and always state the leftover.

Built from 8 past MSRIT papers and the CS54 lecture slides · Units 1, 2 and 3 up to IPv4 addressing · All the best.